RMIT UNIVERSITY

Assignment 2 Industry-Focused

COSC3080|Computer and Internet Forensics

Final Report

Lecturer**: Dr. Sreenivas Tirumala**

Group: A2S12026 7

Student Name - Student ID

Pham Duy Bao - s4118510

Le Chi Bach – s4123419

Table of Contents

Q1. Network forensics from network capture 4

a. Wireshark Filter for Identifying Potential Attack Traffic 4

b. Identifying the Attacker's Host from Wireshark Filter Output 6

Q2: Malware forensics from network capture 8

a. Creation of a Wireshark filter for identifying image files in a network capture with justification. 8

b. Show the HTTP request(s) of the image file(s) and the host name(s) 9

c. Analysis of Suspicious Image File Traffic in Wireshark 10

d. Step-by-Step Process to Export and Verify a Malware Executable File 12

e. Identifying HTTPS/SSL-TLS Traffic and Destination IP Addresses Without Domain Information 19

f. Certificate Issuer Identification of Target Host 23

Q3: Mobile Data Acquisition and Analysis 27

Part 1: 27

a. Create an Android Virtual Device using Android Studio 27

b. Create Contacts and Messages 30

Part 2: 33

c. Data acquisition using ADB tool 33

d. Identify the locations and names of the SMS and Contact databases 35

e. Copy the databases to the computer. 36

f. Access permission issue 37

g. Retrieve the SMSs and the contact details. 37

h. Conclusion 39

Q4. Digital Forensic Analysis and Preparing Investigation Report 39

I. Introduction 42

II. Executive Summary 42

1. Background & Scope. 42

2. Key Findings 43

3. Impact 43
4. Recommendations 43
III. Scope and Objective of the Engagement 44
1. Engagement Scope 44
2. Engagement Objectives 44
3. Changes & Value-Adds 44
IV. Evidence Verification and Integrity 44
Evidence Verification Table 45
V. Findings and Analysis: 45
1. Finding #1: Recovery of Hidden Password File (Medium Risk) 45
2. Finding #2: Hash Identification and Password Recovery (High Risk) 46
3. Finding #3: Cryptographic Artifact Analysis (High Risk) 46
4. Finding #4: Decryption of Financial Evidence File (Critical Risk) 48
V. Summary of the Forensics Report 48
VI. Conclusion and Opinion from the Analysis 49
1. Finding #1 49
2. Finding #2 49
3. Finding #3 49
4. Finding #4 50
VII. Appendix 51
VIII. References 55

Q1. Network forensics from network capture

a. Wireshark Filter for Identifying Potential Attack Traffic

In this task, we are a forensic analyst and have been assigned to investigate a case from one of our clients. It is reported that our client's server had been under a network-based Bot attack (possibly HTTP Flood and Distributed Denial of Service or DDoS), and that the server resources were overwhelmed by requests from various attack machines. It is suspected that the Bot, which is allegedly behind the attack, dynamically changed its user agents, HTTP method types and add a suffix to the end of URLs to penetrate Content Delivery Network (CDN) systems' security. The attack had made the server unreachable to legitimate users.

As suggested by the requirements, the possibility of DDoS will be examined. Therefore, the filter used for this case will be **tcp.flags.syn == 1 and tcp.flags.ack == 0 or http.request.method == "POST" or http.request.method == "GET"**. Applying to the filter, Wireshark will return the list of TCP packets with a syn flag with no corresponding ack flag, as well as the number of HTTP packets with POST and GET request, hoping to find a sign of HTTP flood and SYN flood. To make it easier to visualize, the filter will be broken down into 2 parts: **tcp.flags.syn == 1 and tcp.flags.ack == 0** and **http.request.method == "POST" or http.request.method == "GET".**

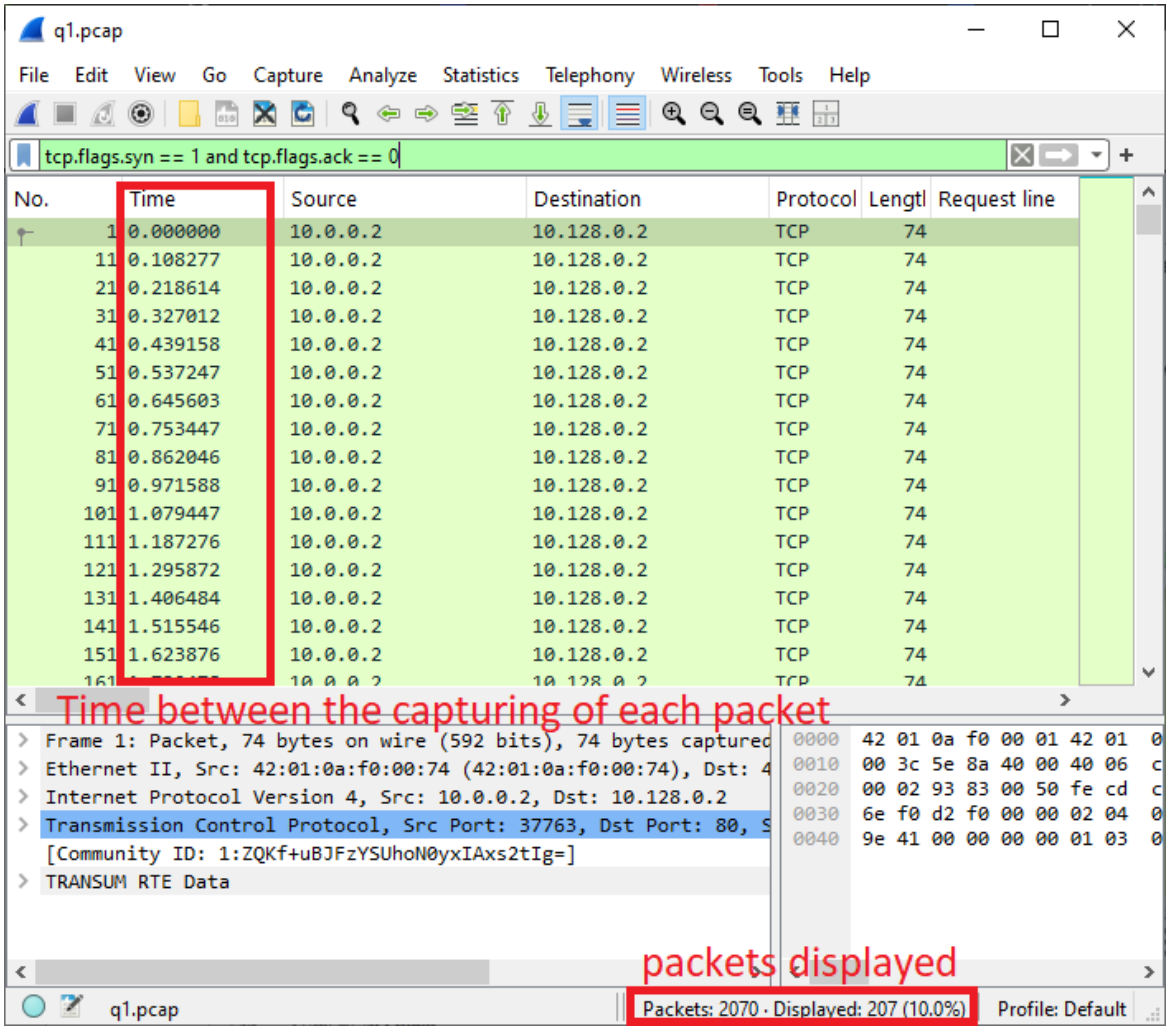


Figure 1: Wireshark filter result for **tcp.flags.syn == 1 and tcp.flags.ack == 0**

As shown in Figure 1, 207 packets matched the first part of the filter. TCP protocol uses a **3-way-handshake** process to ensure a reliable client-server connection. The client side sends a **SYN** message to the server; the server will receive the **SYN number** and reply with its own **SYN** and an **ACK** message. The client will then receive the **SYN-ACK** message and reply with an **ACK**, which the server is expecting to receive. As seen in the image, there are a lot of packets with a **SYN** flag but without an **ACK** flag[1]. The client side didn't send the **ACK** message, so the server must leave an open port on timeout to wait for that response. We can also see that a number of these packets were sent within a second. This process will occupy all available ports of the server, leaving it unavailable for legitimate users. The attackers have used **SYN flood methods** to create the DDoS attack [[2].

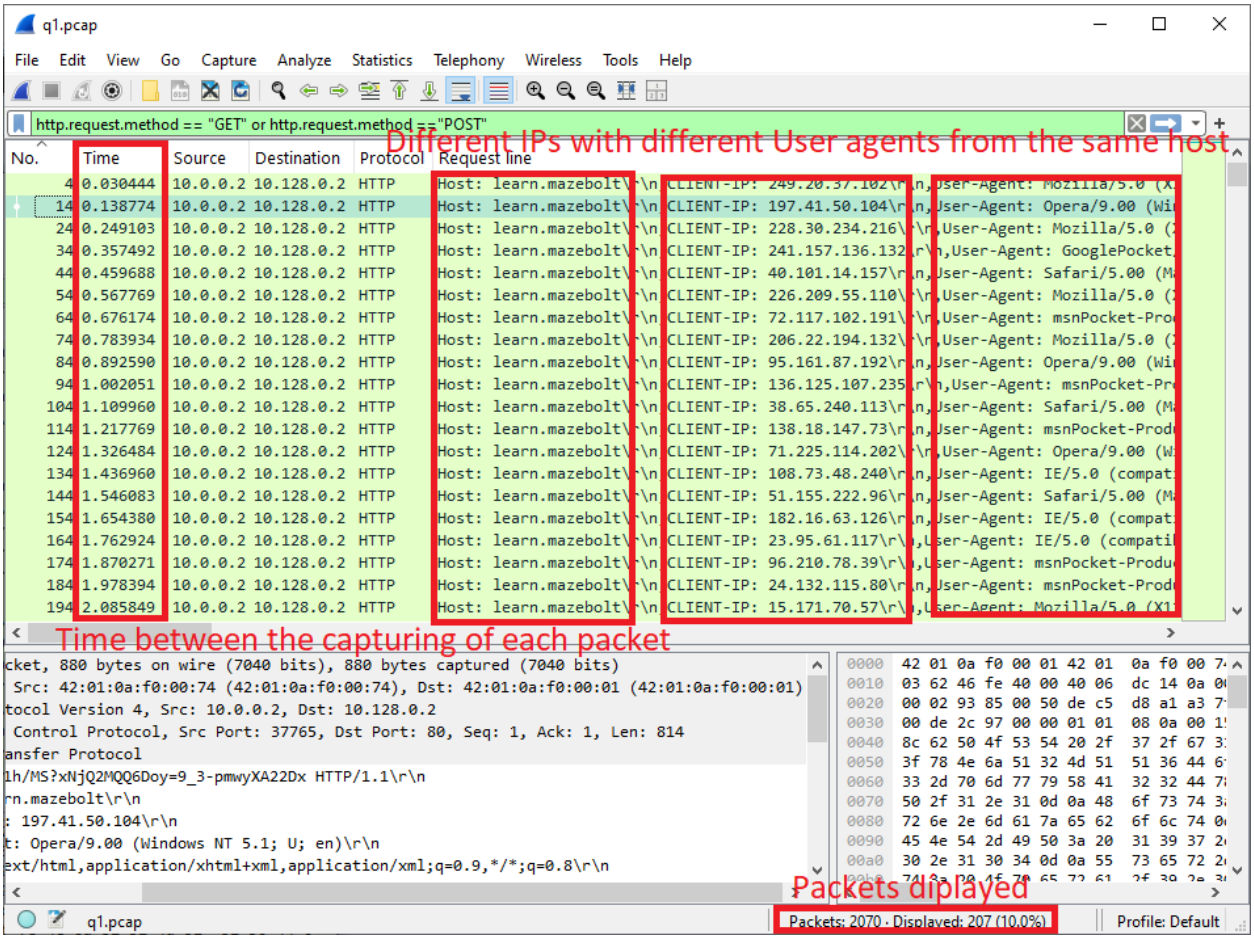


Figure 2: Wireshark filter result for **http.request.method == "POST" or http.request.method == "GET"**

As shown in Figure 2, there are 207 packets matched to the second part of the filter. These packets have a **HTML protocol with POST or GET method**, and there are just a few milliseconds between the capturing of each packet. It also can be observed that the request line follows a similar pattern: **same host, client IP addresses, and User agents changed constantly**. This can be a sign of a **HTTP flood method**. Different IP addresses indicate that this **DDoS attack** uses a botnet, which uses IP spoofing or leverages addresses of devices infected by malware to flood the server with requests. Constantly changing User agents helps the malicious requests blend in with legitimate ones and bypass certain restrictions. [3]

b. Identifying the Attacker's Host from Wireshark Filter Output

Based on the filtered results, it is suspected that **10.0.0.2** was the attacker's IP address. As shown in Figure 2 provided in section a, **10.0.0.2** was the source that sent all the malicious SYN flags and the http requests.

The attacker can also be identified by examining endpoint statistics. This can be done by navigating to the toolbar, selecting Statistics, then Endpoints, and the Endpoints window will be displayed. As shown in Figure 3, in the IPv4 tab, **10.0.0.2** was the source that transmitted 207 packets. Figure 4 presents the result of the TCP tab; **10.0.0.2** was also the source of several packets each transmitted through a different port. Therefore, it can be determined that **10.0.0.2** was the attacker's IP address.

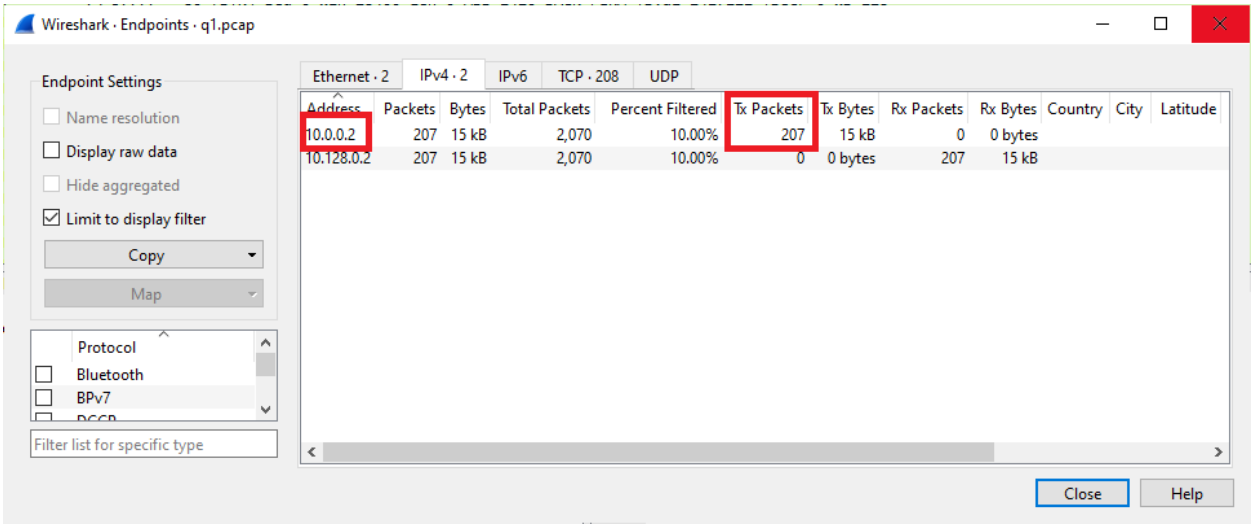


Figure 3: Wireshark Endpoints of IPv4 Transmission

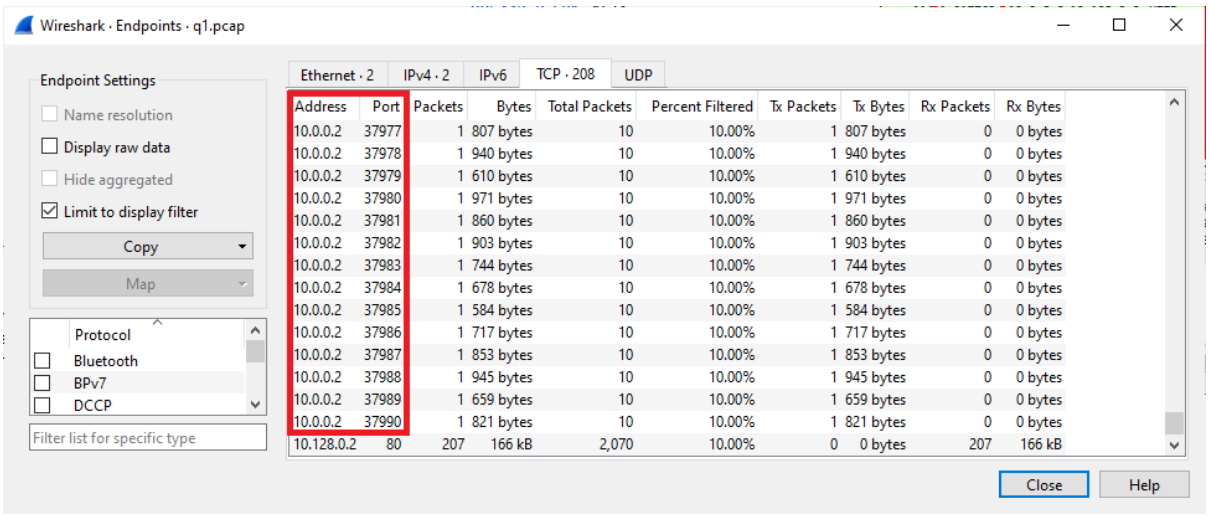


Figure 4: Wireshark Endpoints of TCP Transmission

With the attacker's IP address, a **reverse DNS lookup** can be performed to discover the attacker's hostname.

Q2: Malware forensics from network capture

a. Creation of a Wireshark filter for identifying image files in a network capture with justification.

To identify image files in a PCAP file, we need to look for MIME types such as:

- image/png
- image/jpeg
- image/gif

MIME (Multipurpose Internet Mail Extensions) is a standard used to define the type of data transmitted over a network. Therefore, we will look for file formats, or what we can call Content-types, that are image files.[4]

The command is used: **http.content type contains "image"**

Where:

- **http.content type:** The field in the HTTP response header indicates the type of data the server is returning
- **Contains “image”:** Filter packets whose Content-Type value contains the word "image"

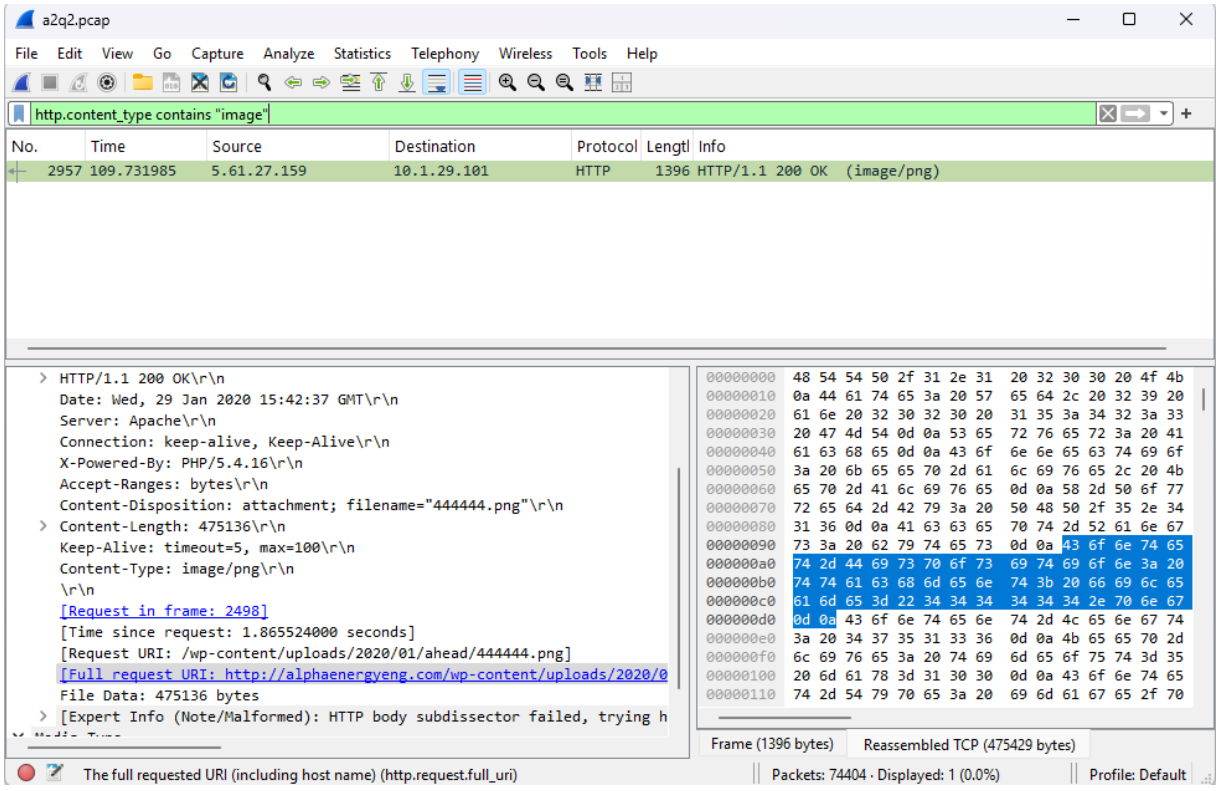


Figure 5: Wireshark filter results showing HTTP image Content-Type

b. Show the HTTP request(s) of the image file(s) and the host name(s)

To view the request, we go to the Hypertext Transfer Protocol section. However, since this is a response, we will go to frame 2498 as indenticated in the notification (as shown in figure 6)

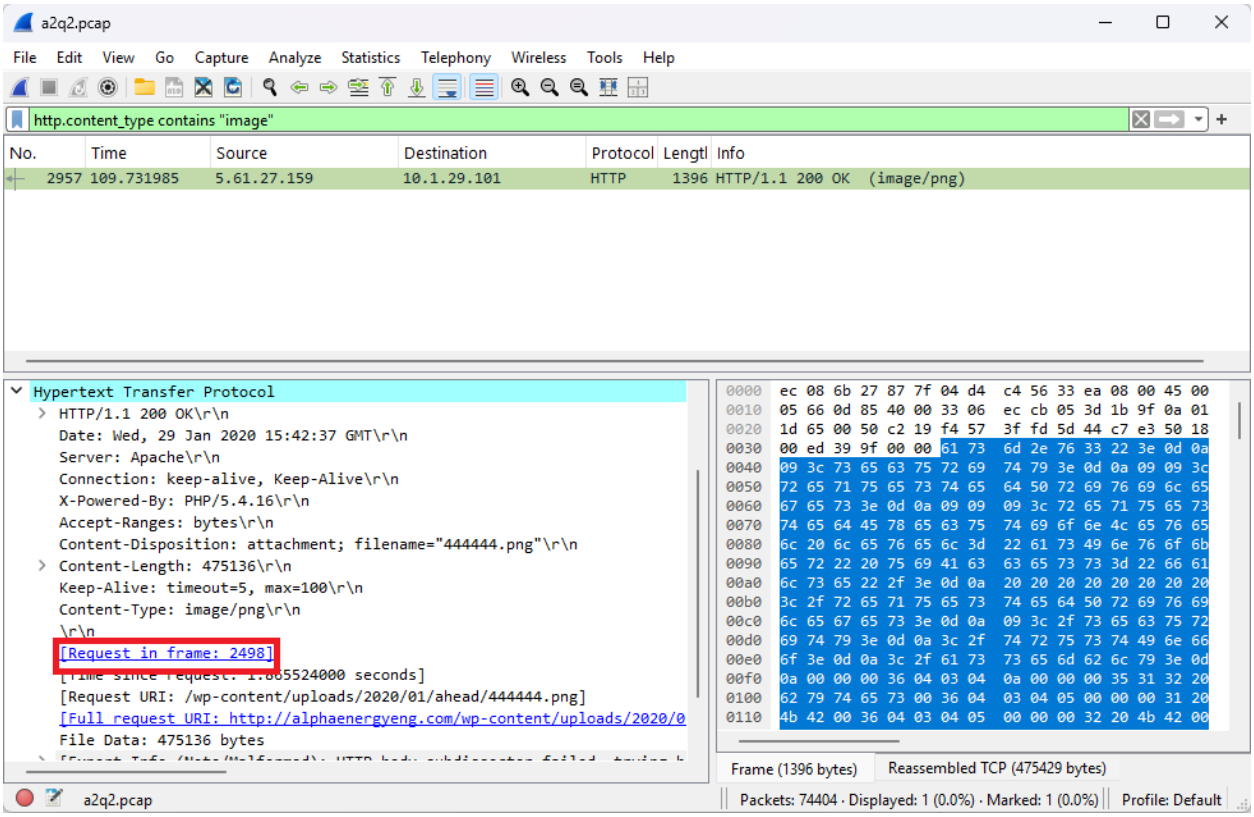


Figure 6: HTTP response showing image file and corresponding request frame in Wireshark

Upon examining frame 2498, detailed request information was identified, including the following data:

- **Request Method**: GET
- **Request URL:** /wp-content/uploads/2020/01/ahead/444444.png
- **Request Version:** HTTP/1.1
- **Host name:** alphaenergyeng.com

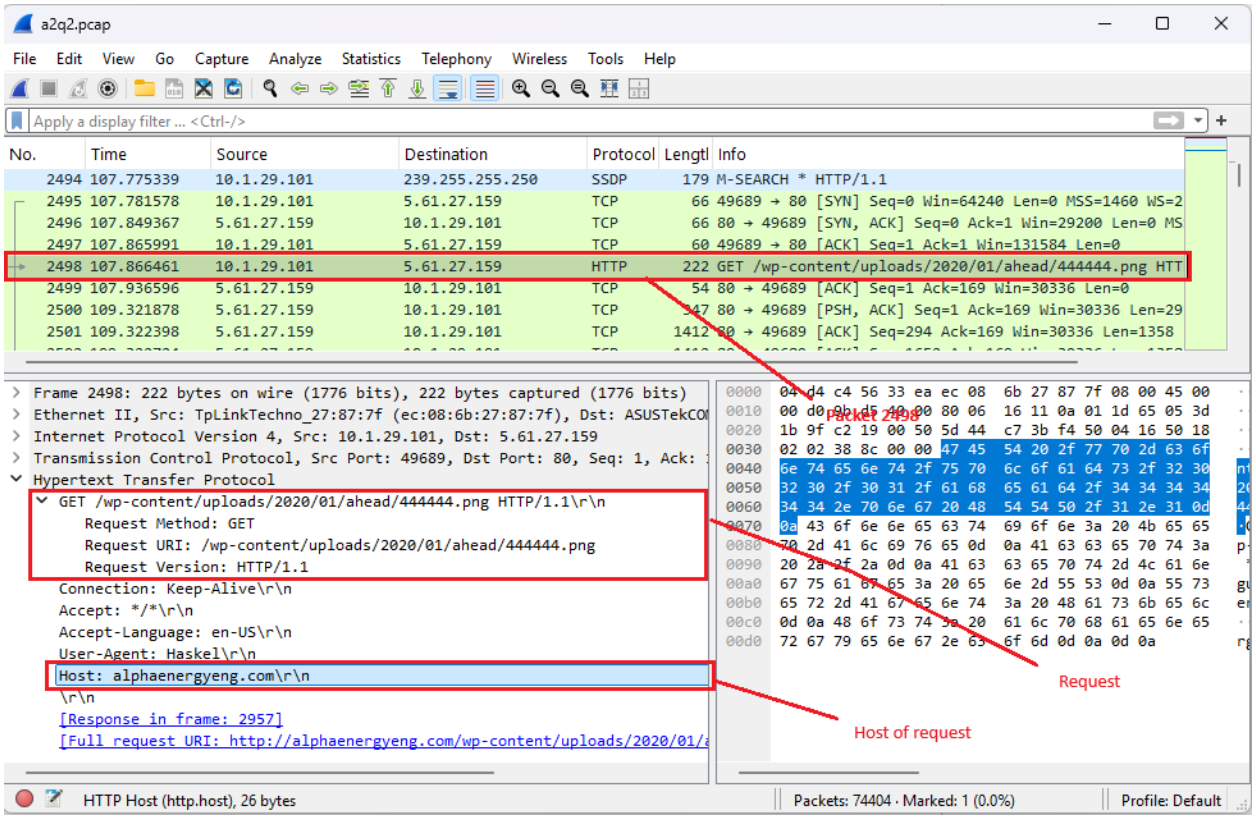


Figure 7: HTTP request and host name of the image file in Wireshark

c. Analysis of Suspicious Image File Traffic in Wireshark

Several characteristics were identified that make this file appear suspicious, including:

- Abnormal file characteristics
- MIME type mismatch
- Suspicious URL/domain structure

After conducting a general assessment, the following observations were made:

1. Abnormal File Characteristics

The file size was approximately **475,136 bytes (0.45 MB)**, which is relatively normal for an image file. Therefore, the file size alone does not strongly indicate malicious activity.

2. Suspicious URL/Domain Structure

Based on the URL:

http://alphaenergyeng.com/wp-content/uploads/2020/01/ahead/444444.png

several suspicious indicators were identified. The unusual /ahead/ directory within the WordPress upload structure may suggest an attempt to conceal malicious content, which is consistent with a **Masquerading** technique. In addition, the

filename 444444.png appears meaningless and non-descriptive, which is commonly observed in malware delivery campaigns.

Furthermore, the file was delivered over unencrypted **HTTP** rather than **HTTPS**. The absence of transport encryption is often associated with compromised websites or temporary “ghost” domains used for malware distribution.

3. MIME Type Mismatch

Although the file was identified as a PNG image based on its MIME type and file extension, further analysis indicated that it was likely an executable (exe) file. This conclusion was supported by the presence of the **“MZ”** magic bytes identified during packet inspection (as shown in figure 8). The MZ header is a well-known signature for Windows Portable Executable (PE) files and confirms that the file content does not match its claimed PNG format.

4. Conclusion

In summary, multiple indicators suggest that the file is potentially malicious. These indicators include the suspicious directory structure and filename, the use of unencrypted HTTP traffic, and the mismatch between the MIME type and the actual file content identified through magic byte analysis.

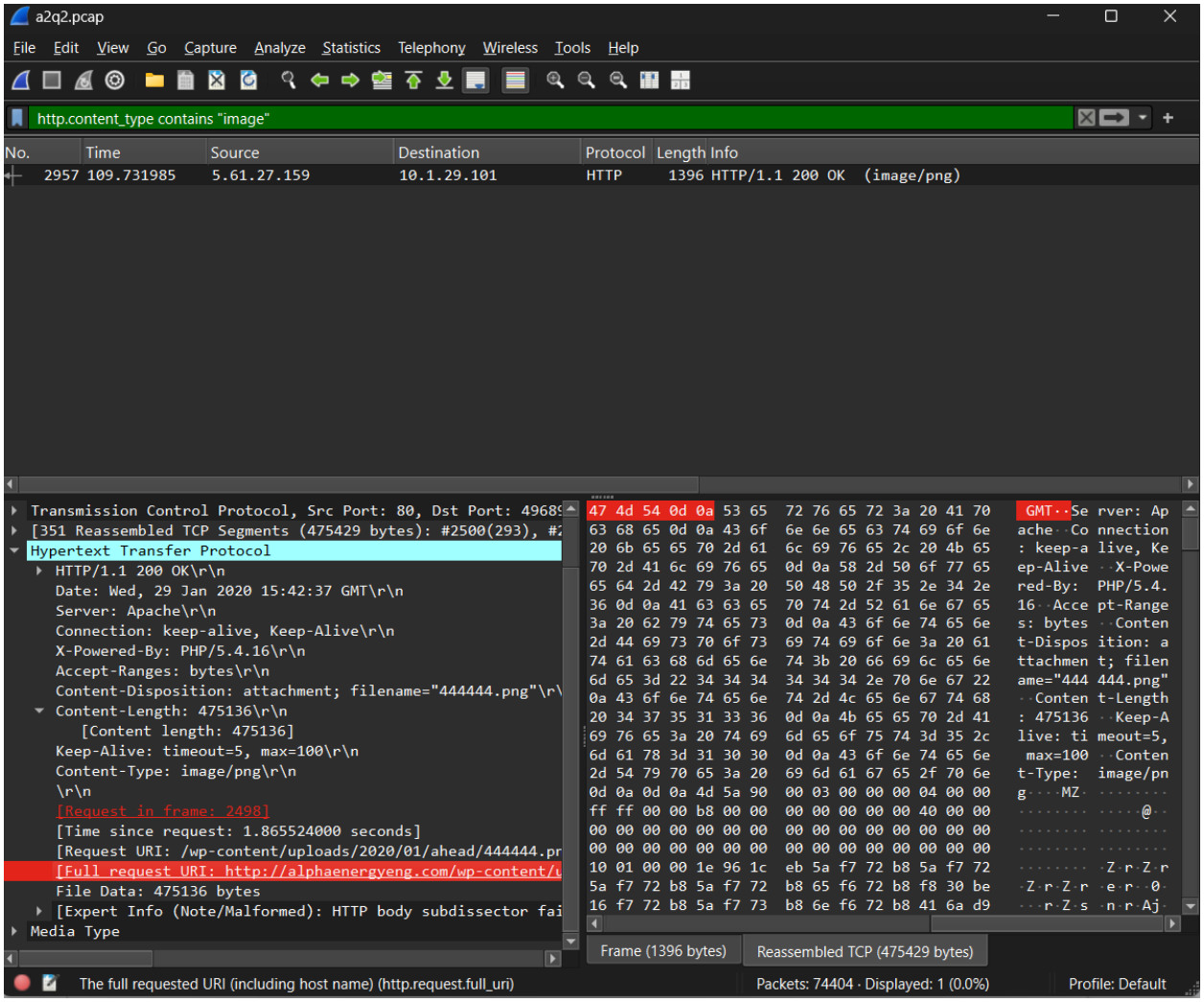


Figure 8: Magic Byte Analysis Revealing Executable Content Through “MZ” Header

d. Step-by-Step Process to Export and Verify a Malware Executable File

Because this is very likely a malware file, to be safe I moved it from my machine to Kali Linux in a virtual machine. This helps isolate the environment to prevent accidentally running an unknown file that could affect the real computer system. To extract the file from network packets, follow these steps:

Step 1:

In Wireshark, open the object extraction menu by selecting File → Export Objects → HTTP…, since the suspicious file was transmitted through the HTTP protocol.

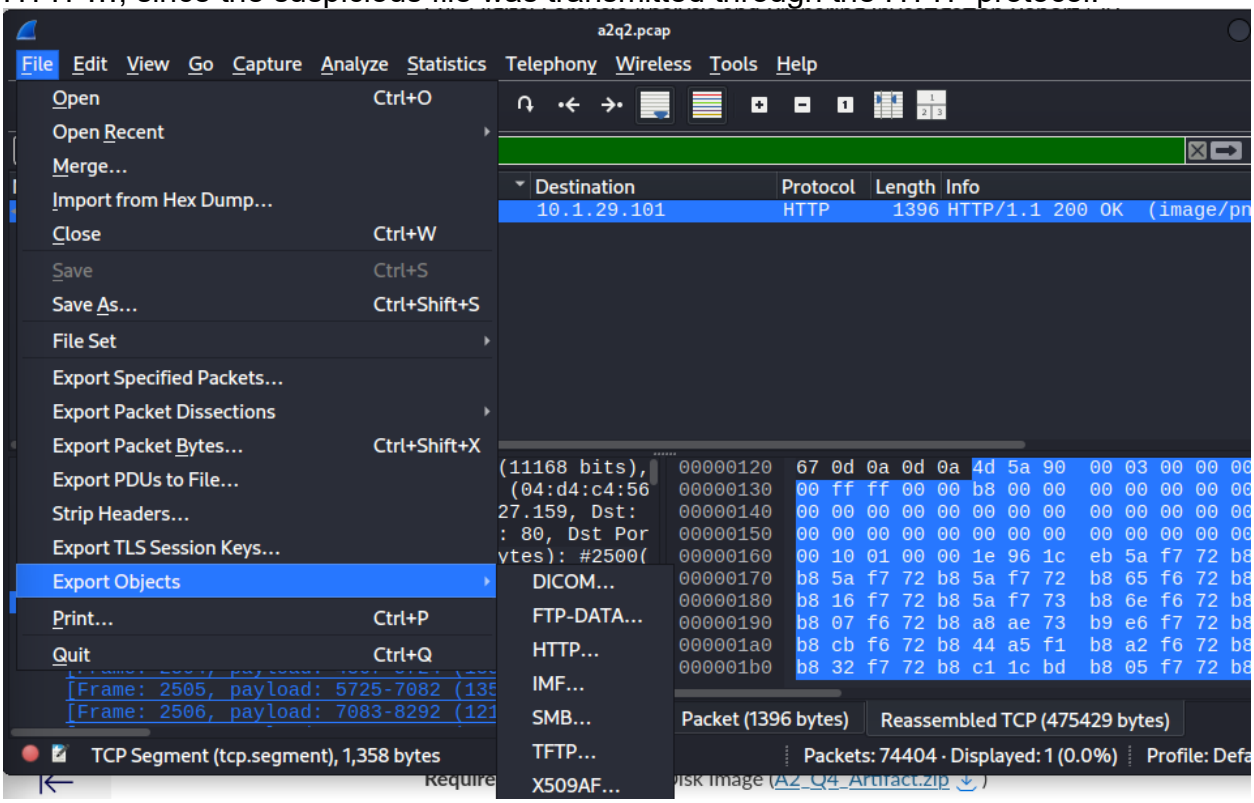


Figure 9: Exporting HTTP Objects from Wireshark

Step 2: Filter and identify the target file

In the Wireshark Export HTTP object list window, the "**Text Filter**" box can be used for quick searching. Enter: ***http.content_type contains "image"***. Search for the line containing the following information:

- Packet: 2957
- Hostname: alphaenergyeng.com
- Filename: 44444.png
- Size: 475kB.

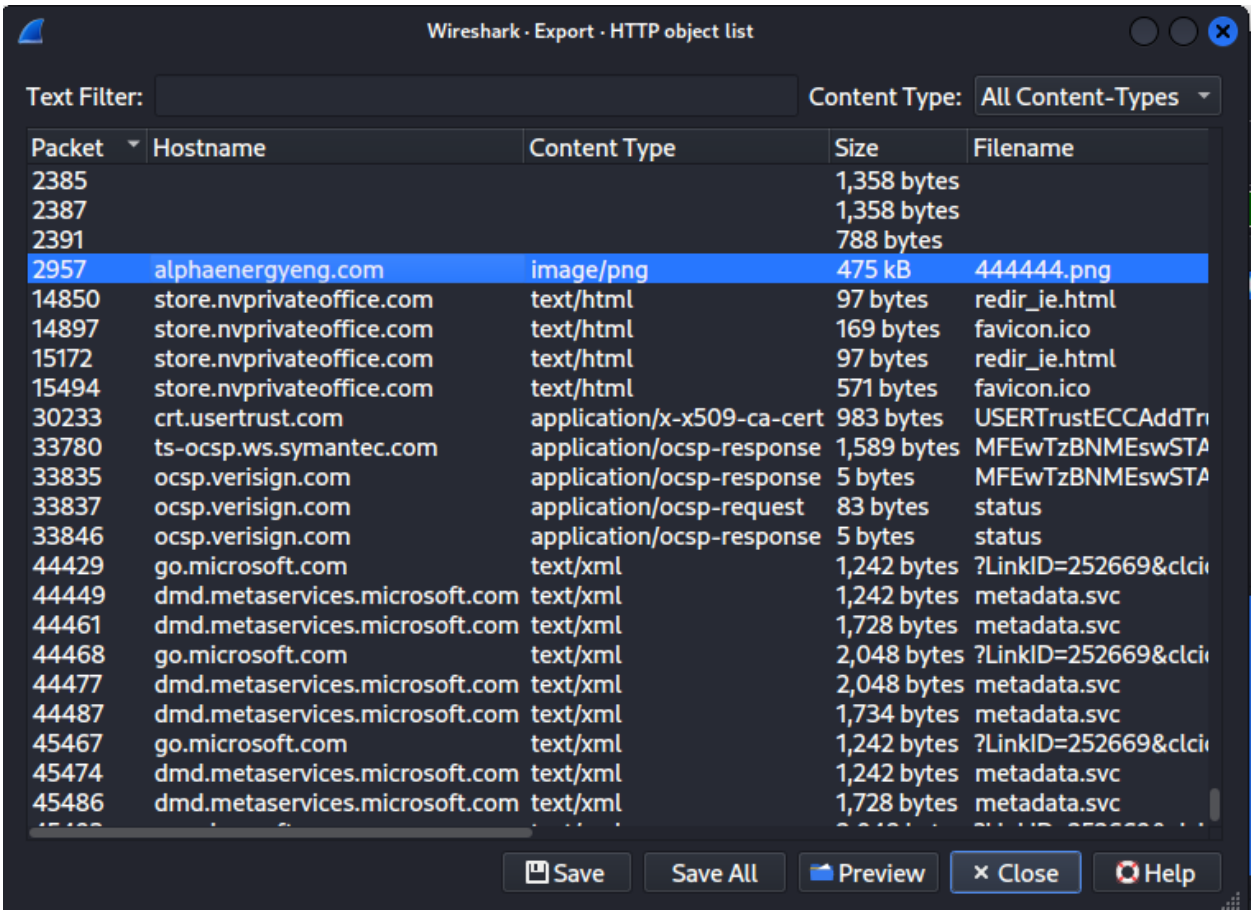


Figure 10: Identification of Suspicious Image File (44444.png) in Wireshark HTTP Objects

Step 3: Save the file to an isolated environment:
Select the line corresponding to the file 44444.png and click **Save**. Then Create and select the folder ***/home/kali/Desktop/suspect_malicious_file*** to store the file for management and isolation purposes.

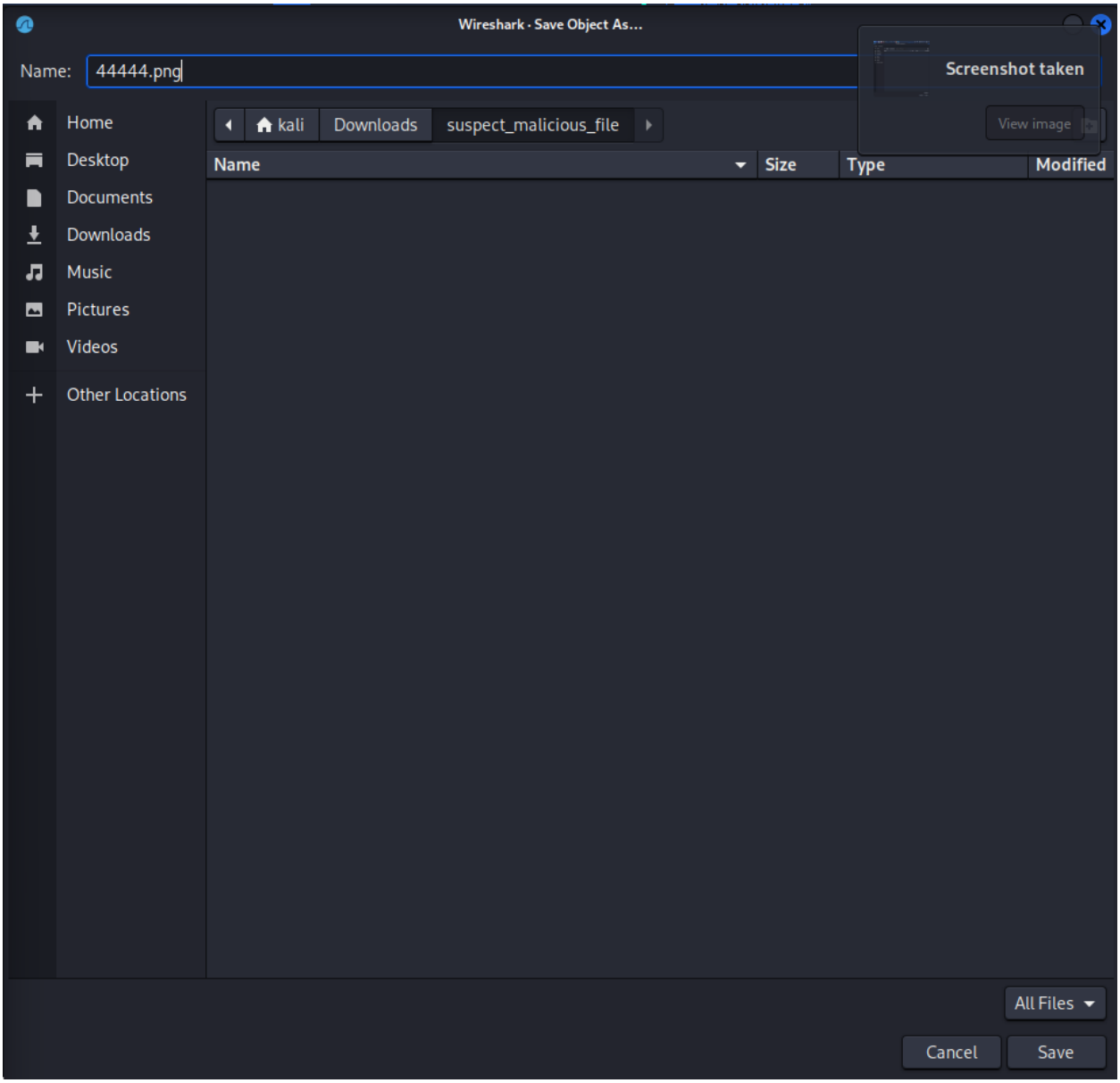


Figure 11: Saving Suspicious Image File (44444.png) to Local Machine

Step 4: Verify the malware file in Virustotal

Since this is a file, we select the **File** option in VirusTotal and choose the file we want to verify (as shown in figure 12). Then, navigate to ***/home/kali/Desktop/suspect_malicious_file/44444.png*** and select the file



Figure 12: Virustotal homepage

After selecting the file, VirusTotal analyzes it and provides results. VirusTotal conducts a scan using multiple engines, verifying the document with more than 70 antivirus tools. The summary shows that 58 out of 71 security vendors flagged this file as malicious (as shown in figure 13). The Detections section shows that this file is very likely a Trojan (as shown in figure 14). Additionally, the Details section reveals that it is an executable file (.exe), confirming that our initial assessment was correct (as shown in figure 15).

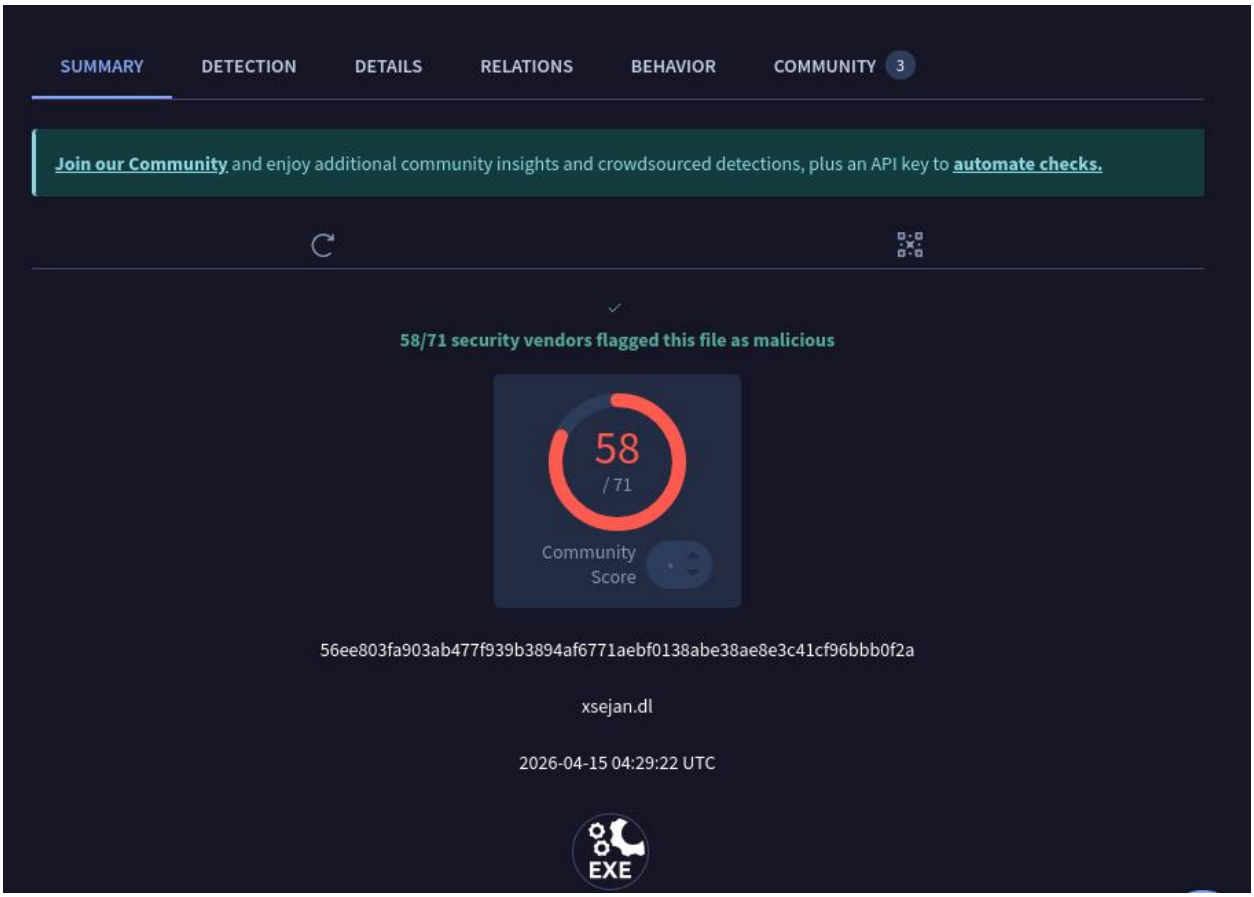


Figure 13: VirusTotal Summary Showing 58/71 Security Vendors Flagging the File as Malicious

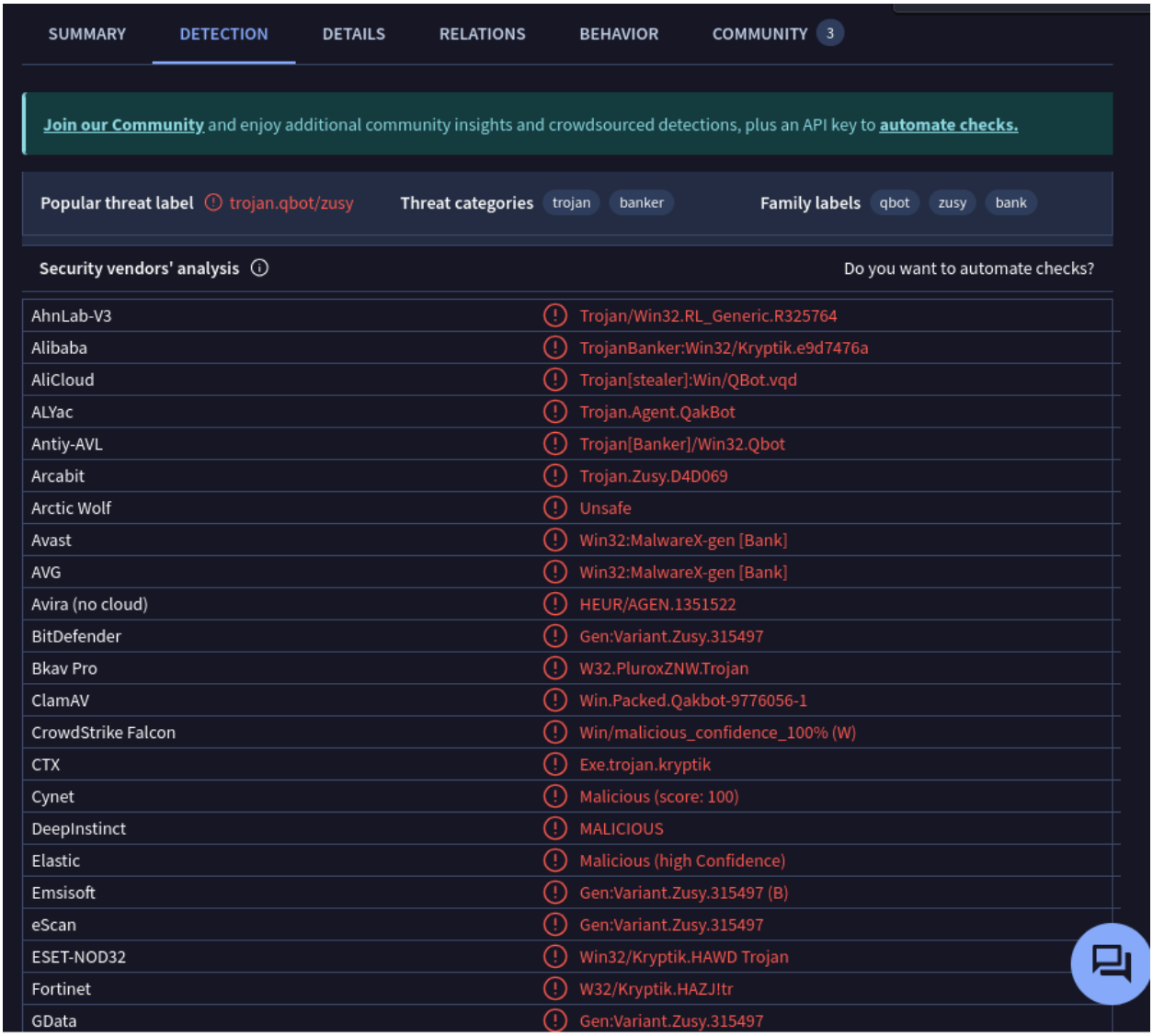


Figure 14: VirusTotal Detection Results Identifying the File as a Trojan

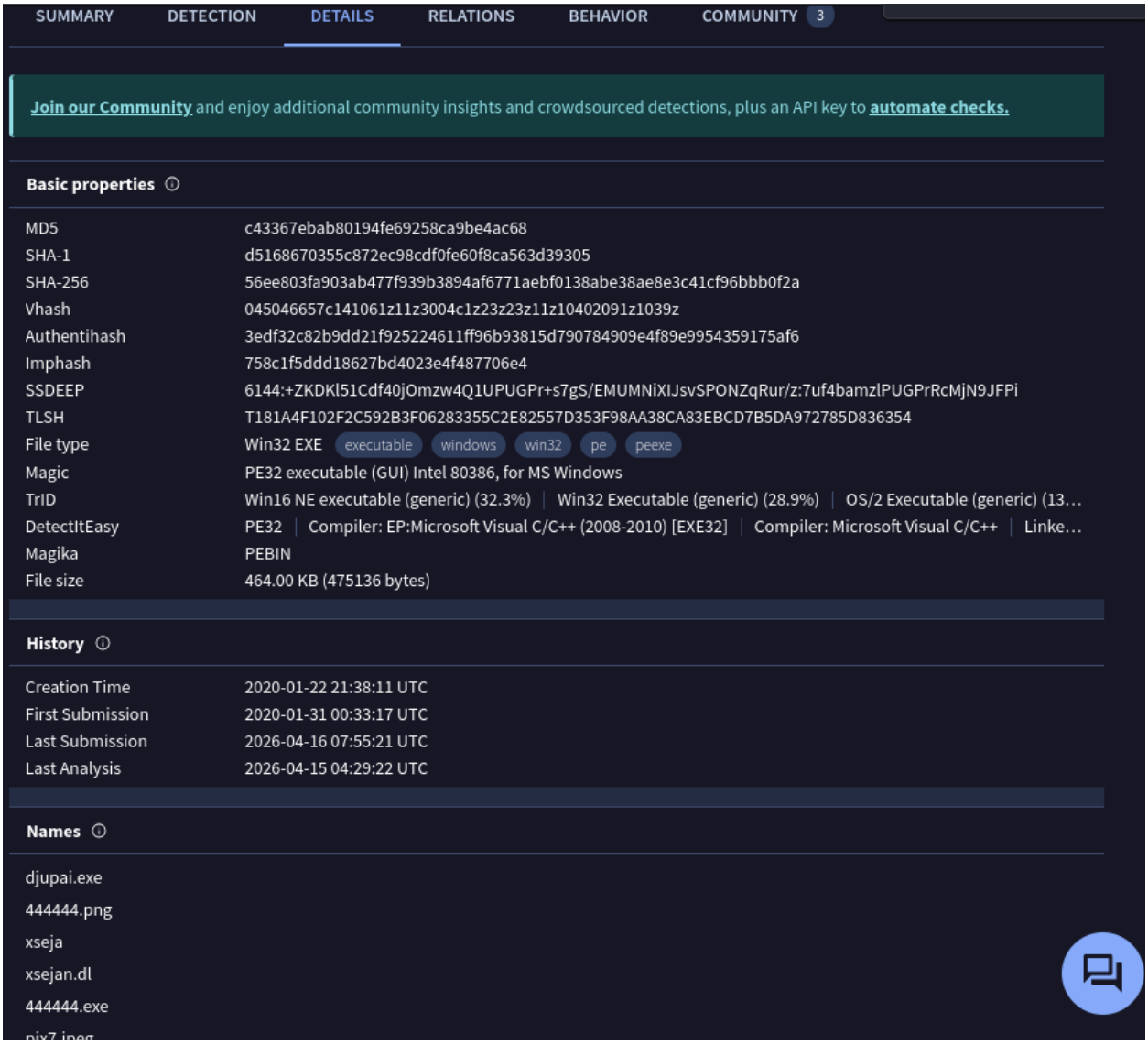


Figure 15: VirusTotal Details Section Revealing the File as an Executable (.exe)

e. Identifying HTTPS/SSL-TLS Traffic and Destination IP Addresses Without Domain Information

The filter ***http.request*** shows URLs from HTTP requests, whereas ***tls.handshake.type eq 1*** detects domain names associated with HTTPS or SSL/TLS communication. When examining Windows web traffic, this filter may capture HTTP requests via **UDP port 1900**, which is related to **Simple Service Discovery Protocol (SSDP)**. To exclude SSDP traffic, the filter can be refined to: ***(http.request or tls.handshake.type eq 1) and !(ssdp)***. From there, **TLS handshake packets** can be identified and their **Server Name Indication (SNI)** values examined. The **SNI** field typically reveals the **domain**

name being accessed. However, if a **TLS packet lacks an SNI**, this may indicate communication conducted directly through **IP addresses** rather than domain names.

Once the filter was applied, the resulting traffic appeared as shown below:

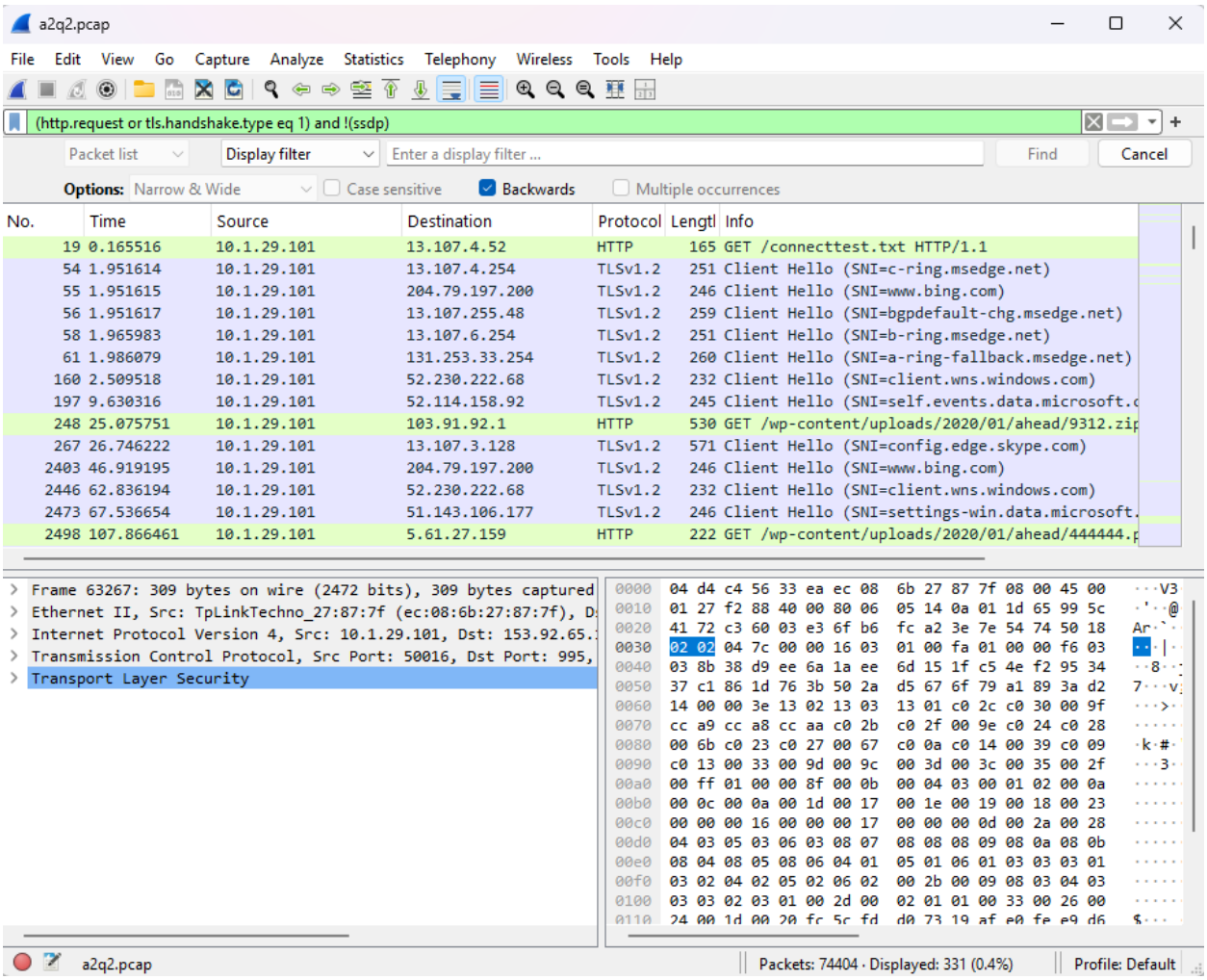


Figure 16: TLS Handshake Results Showing HTTPS Traffic Without Domain Names

The **filter (http.request or tls.handshake.type eq 1) and !(ssdp)** capture both HTTP requests and TLS Client Hello messages while excluding SSDP traffic. However, it doesn't differentiate between TLS traffic with and without Server Name Indication (SNI). Therefore, it still includes traffic containing domain names, making it difficult to specifically identify HTTPS/SSL/TLS connections to servers without associated domain names. Therefore, the filter "***tls.handshake.type == 1 and !tls.handshake.extensions_server_name***" can be helpful, where

- **tls.handshake.type == 1**: Filter TLS Client Hello packets (initiating HTTPS connections)

- **!tls.handshake.extensions_server_name**: Filters packets that do not contain a domain name (no SNI)

Use the filter and we will obtain the outcomes as follows:

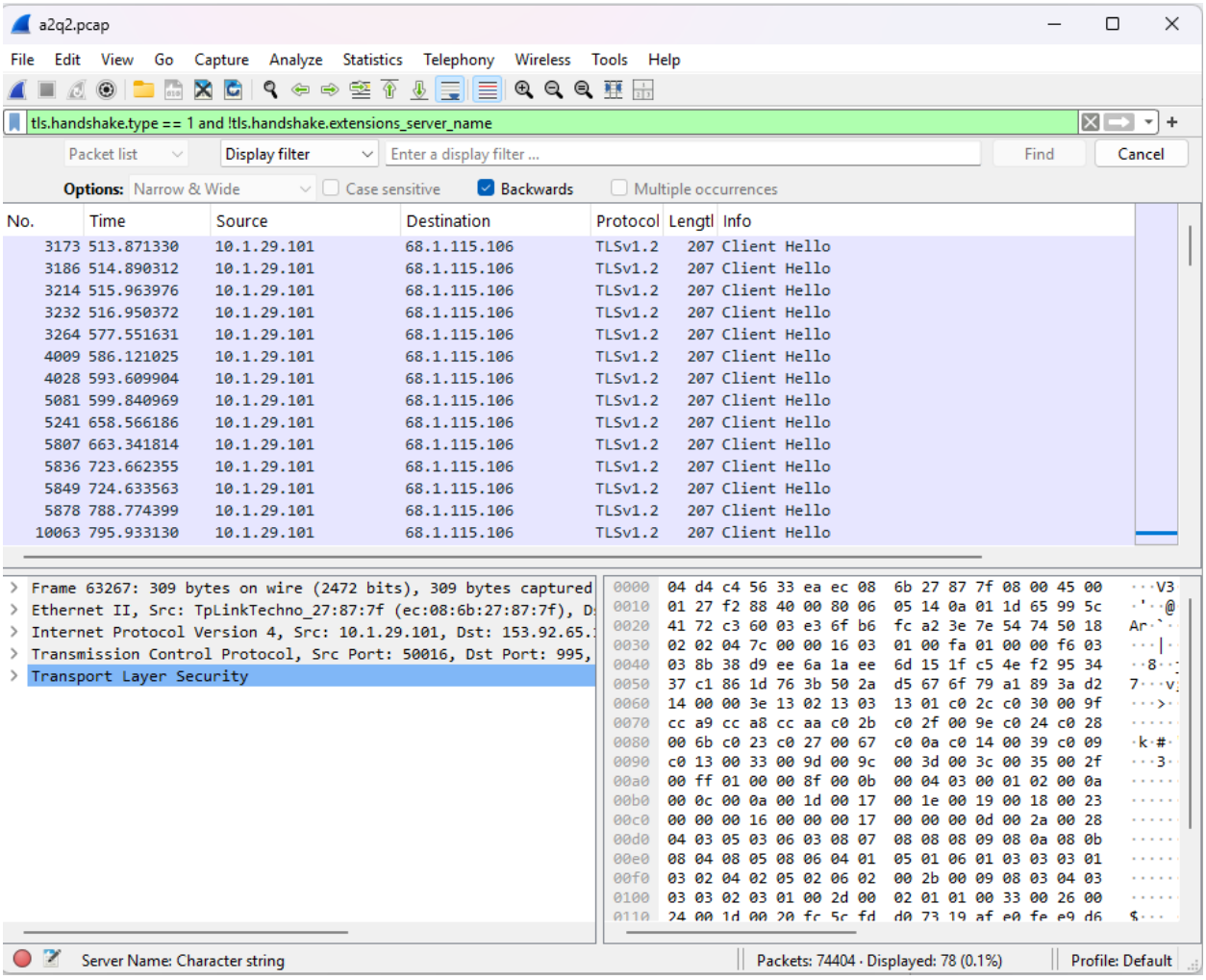


Figure 17: TLS Client Hello Packets Without Server Name Indication (No SNI) Detected in Wireshark

As the outcome might include several packets with the same destination IP addresses, it is essential to isolate unique destination IPs for easier analysis

To achieve this:

- Navigate to the **Statistics** menu on the toolbar (as shown in figure 18)
- Select **Endpoints** from the dropdown menu
- In the Endpoints window, switch to the **IPv4** tab (as shown in figure 19)

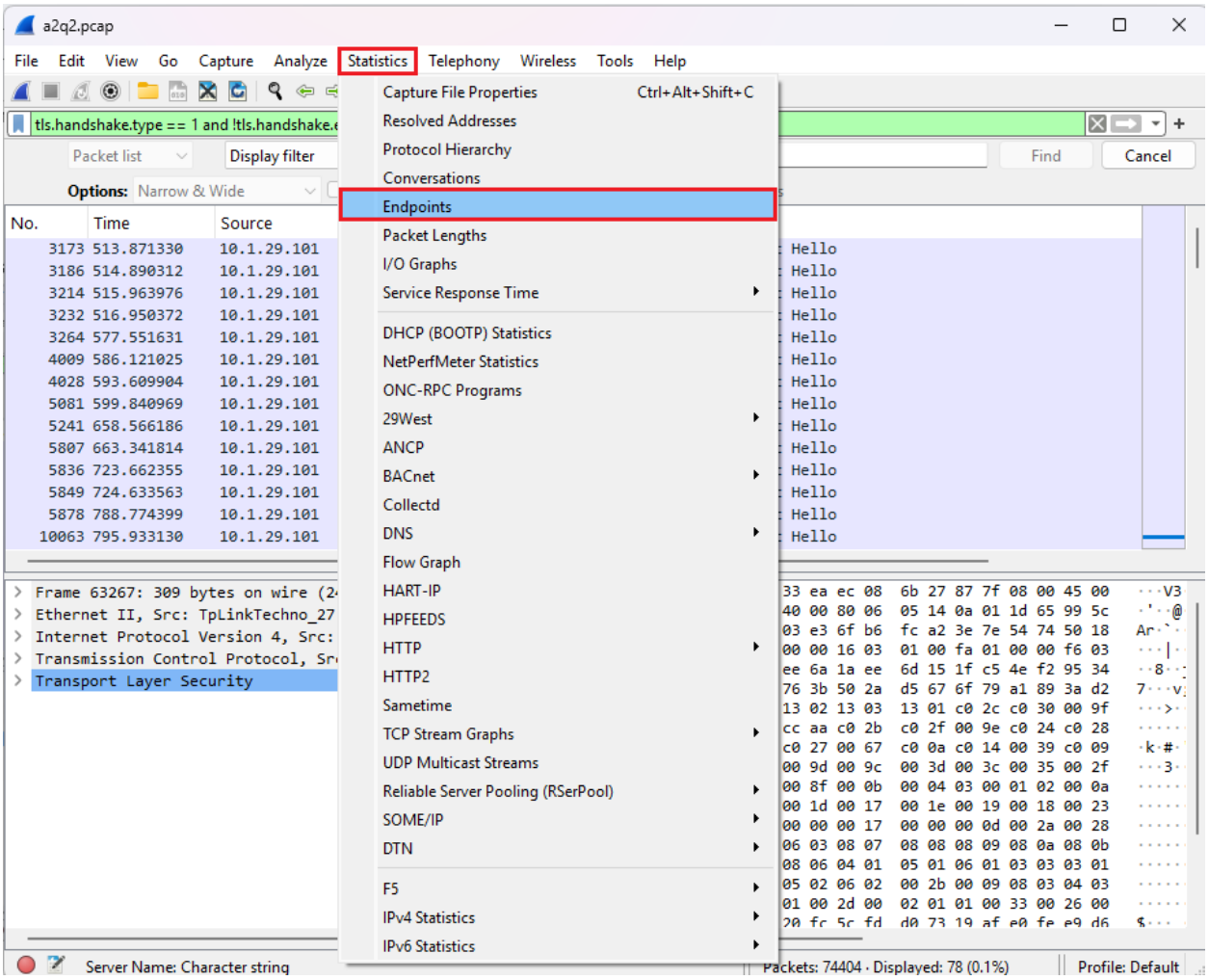


Figure 18: Opening the Statistics Menu in Wireshark

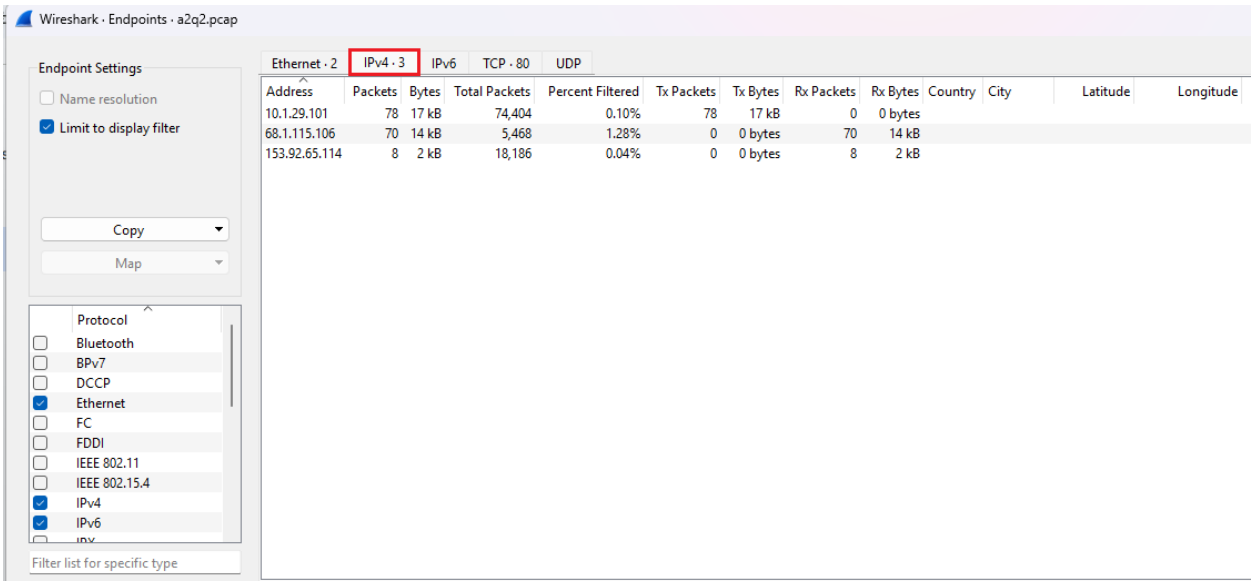


Figure 19: IPv4 Endpoint Statistics Showing Unique IP Addresses

After filtering unique IP destinations, we receive the following results: **10.1.29.101, 68.1.115.106, 153.92.65.114**, while 10.1.29.101 is excluded because it is the source.

f. Certificate Issuer Identification of Target Host

To explore these two IP addresses more deeply, we need to gather details regarding the certificate issuer of the questionable server we identified earlier. Wireshark has a filter designed to capture specific network traffic related to IP addresses and SSL/TLS handshake events, namely `***ip.addr == <IP address>` and `tls.handshake.type ==11***`. The `<IP address>` field contains the actual IP address to be investigated, and `tls.handshake.type eq 11` is used to filter TLS handshake packets containing the server's certificate.

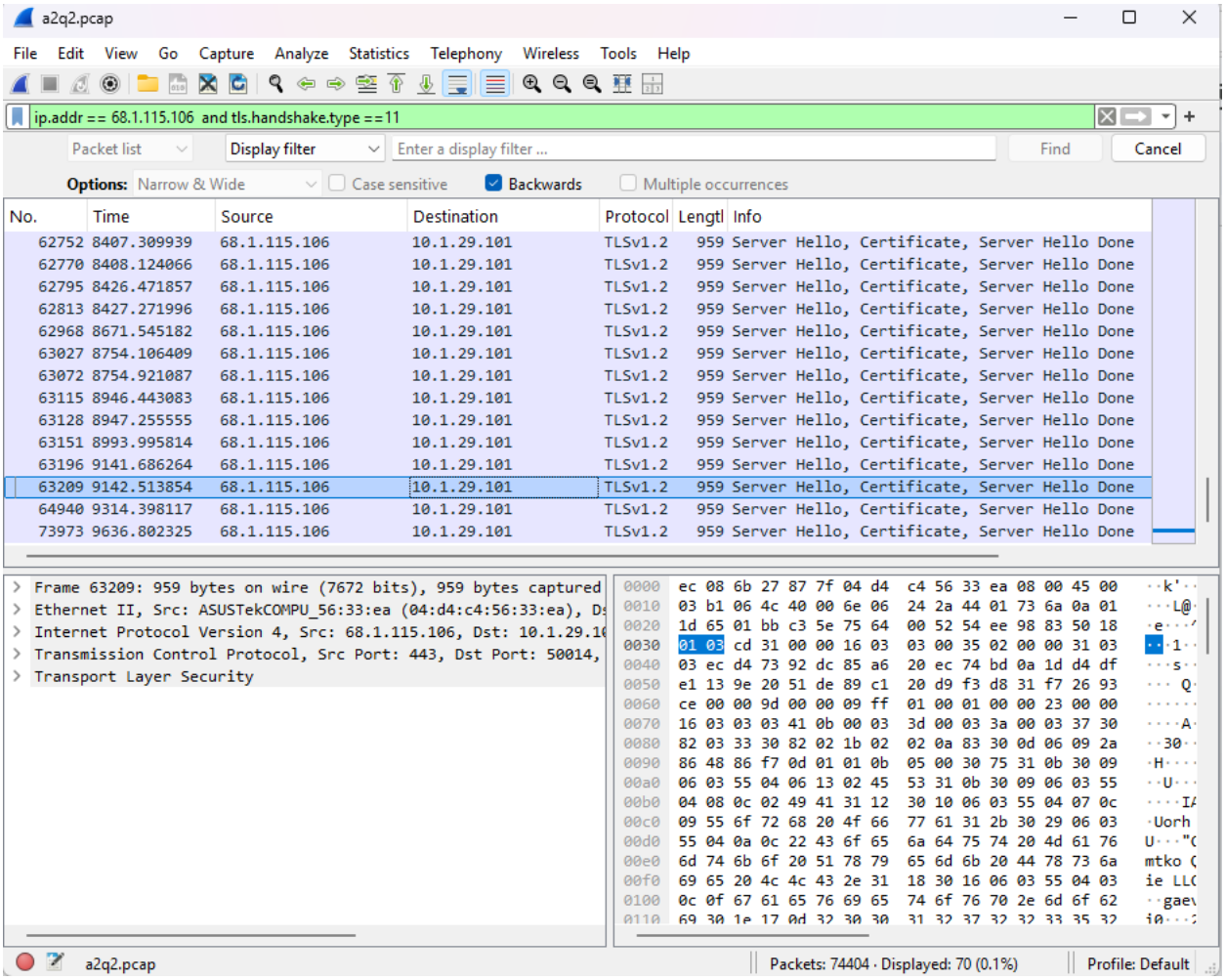


Figure 20: Wireshark Filter Result of TLS Certificate Messages for 68.1.115.106

To find the details of the certificate issuer, we need to open the **Transport Layer Security (TLS) → TLSv1.2 Record Layer: Handshake Protocol: Certificate → Handshake Protocol: Certificate → Certificate →Certificate (shortened) → signedCertificate → issuer**. Then the Subject (RDNSequence) section will appear - This is the "subject identifier" information of the certificate (server) as shown in figure 21.

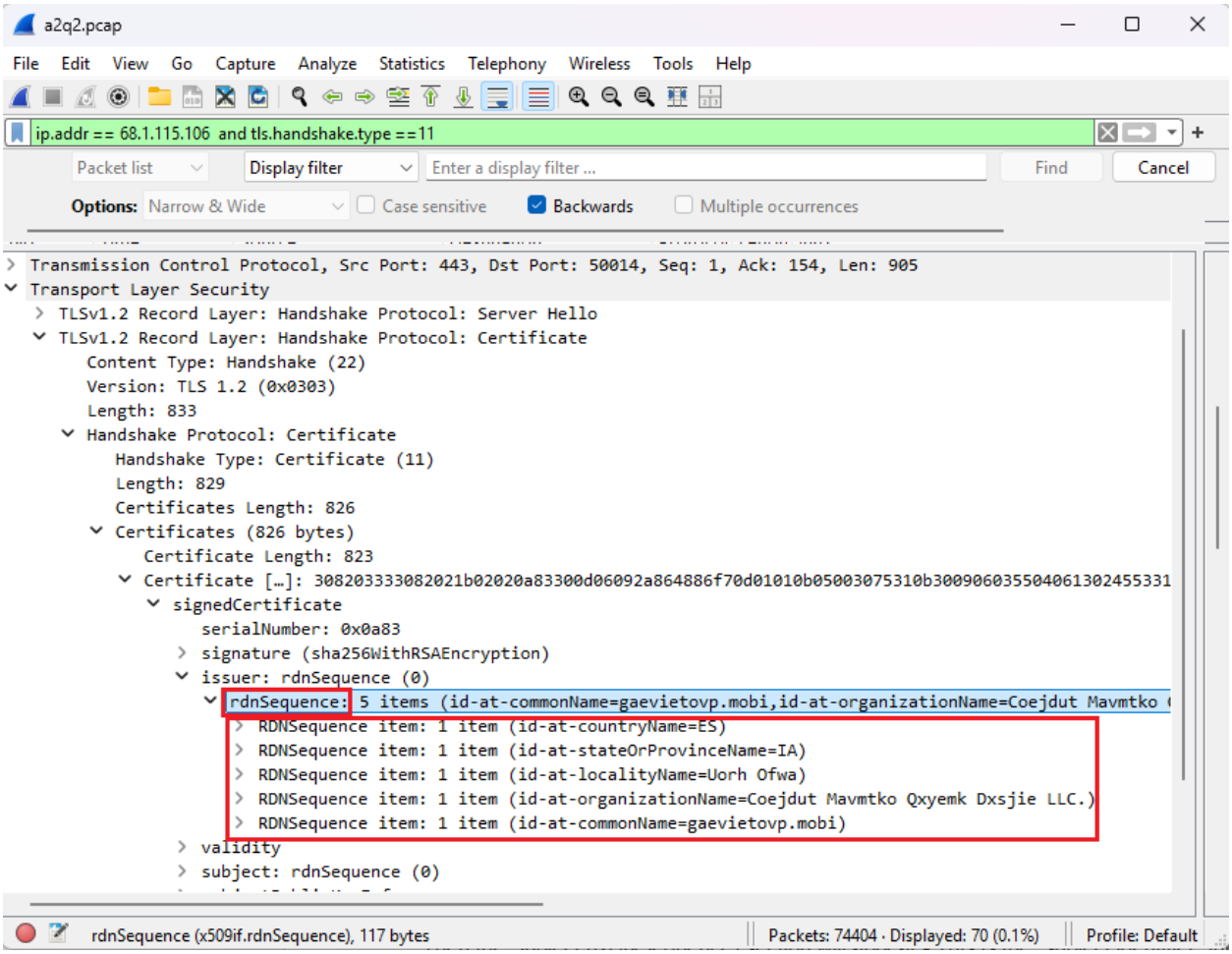


Figure 21: TLS Certificate Details for IP Address 68.1.115.106 in Wireshark

The result of certification of server is:

- **Common Name (CN):** gaevietovp.mobi
- **Organization (O):** Coejdut Mavmtko Qxyemk Dxsjie LLC
- **Locality (L):** Uorh Ofwa
- **State/Province (S):** IA
- **Country (C):** ES

The TLS 1.2 server certificate observed during the handshake displayed unusual and potentially forged subject attributes, such as unverifiable organization names and locations. The domain name "**gaevietovp.mobi**," along with invalid geographic and organizational details, suggests the certificate may have been self-signed or not issued by a trusted Certificate Authority, raising potential security concerns.

Furthermore, in fact, modern CAs (such as Let's Encrypt, DigiCert) use serial numbers up to 20 octets (40 hexadecimal characters) that are randomly generated, as described in section 4.1.2.2. The serial number of RFC 5280 (a reference to the X.509 digital cer-

tificate structure) [[5]. Therefore, a serial number as short as 2 bytes, like 0x0a83, is evidence that it was not generated by a standard CA process, but by an automated script or tool like OpenSSL with simple default configuration.

After applying a similar certificate search method to IP address **153.92.65.114**, we can see that the serial number is correctly written as **0x07a49f1f28dc709dbe7e5fd3f7ee4b3b** (40 hexadecimal characters) and the certificate information includes:

- **Common Name (CN):** DigiCert SHA2 Secure Server CA
- **Organization (O):** DigiCert Inc
- **Country (C):** US

This is a certificate issued by DigiCert – one of the largest and most reputable Certificate Authorities (CAs) in the world.

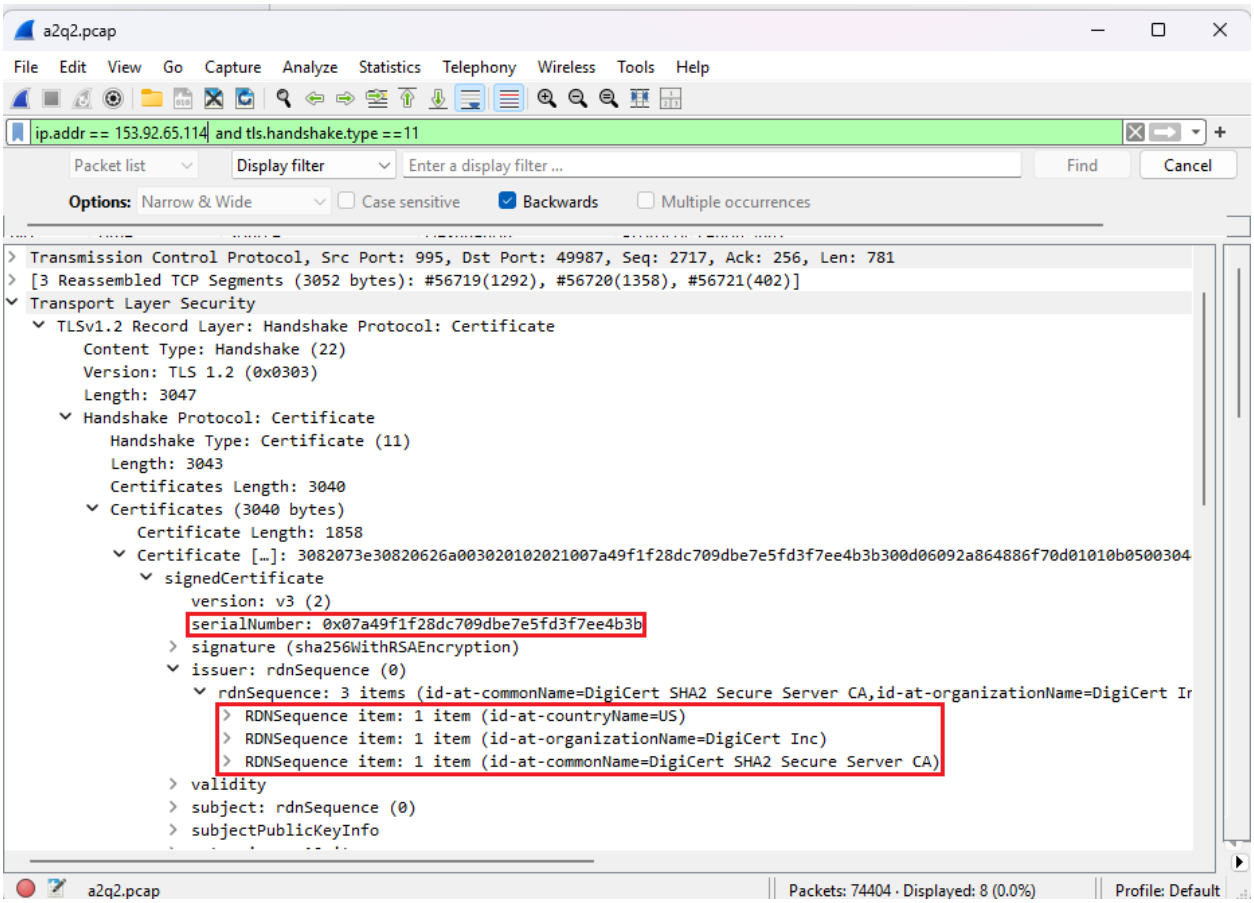


Figure 22: TLS Certificate Details for IP Address 153.92.65.114 in Wireshark

\

Q3: Mobile Data Acquisition and Analysis

Part 1:

a. Create an Android Virtual Device using Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Android app development. It provides an environment to develop Android devices and emulators in real-time[4]. In this task, we will create a Virtual Device using Android Studio and practice Mobile Data Acquisition skills and provide analysis based on our findings.

After installing Android Studio, a welcome window like this will be displayed. Select **More Actions** > **Virtual Device Manager** to open the Device Manager window. (As shown in figure 23)

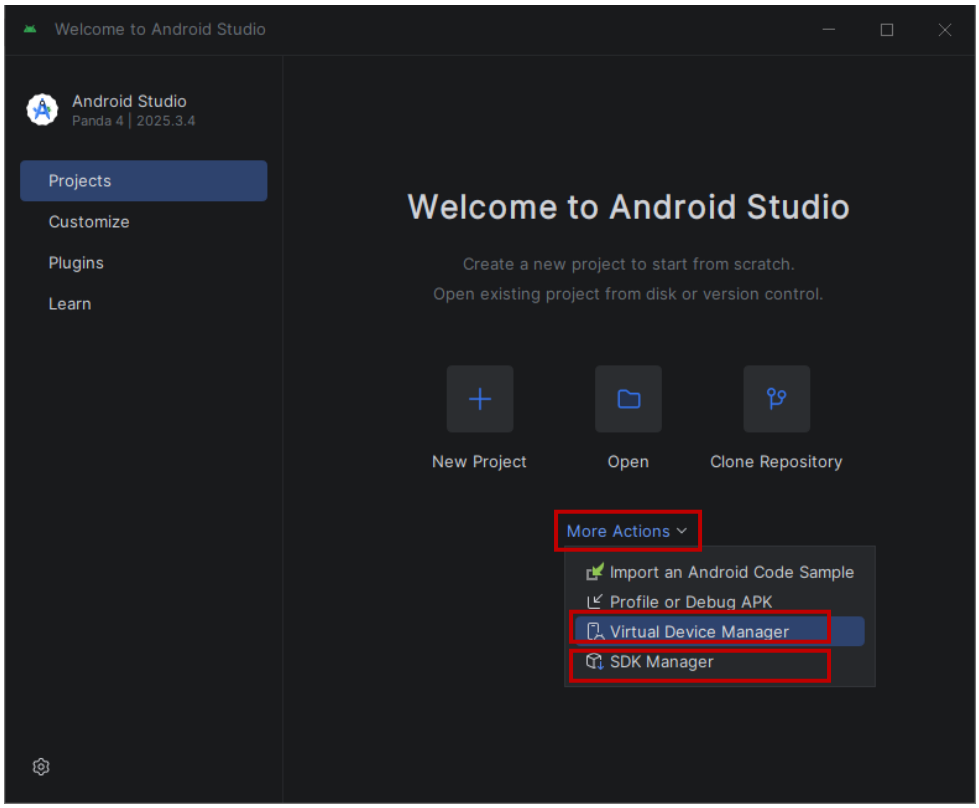


Figure 23: Android Studio Welcome window

This is the Device management window. To start creating an emulator, click on **“Create virtual device…”** in the middle of the window or the **“+”** in the toolbar. Either way, it will open the **Add Device window** (as shown in figure 24).

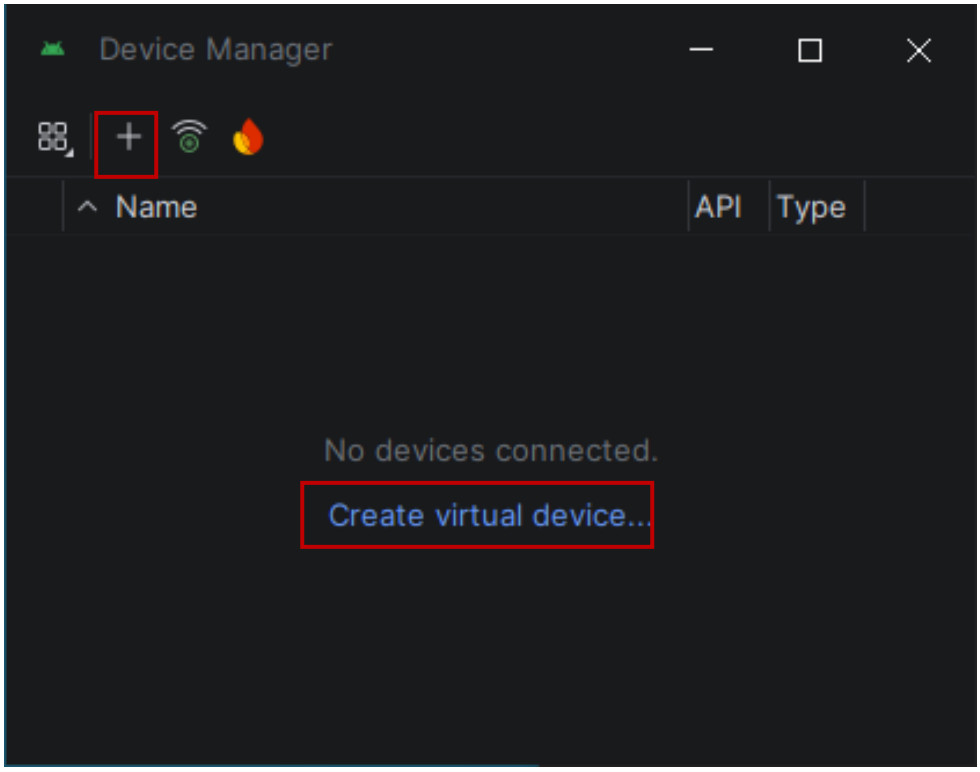


Figure 24: Android Studio Device Manager

For this task, the **Pixel 6 Pro** device will be used as we adopted the instructions from the workshop document. Pixel 6 Pro is an **up-to-date** android device (released in 2021). It also has a **6.7” screen** which will make the visualization of this assignment better. Other devices like **Pixel 4 XL** or **Pixel 7 Pro** can also be used for their large screen, in **consideration of the available resources** of the device on which the virtual device will be

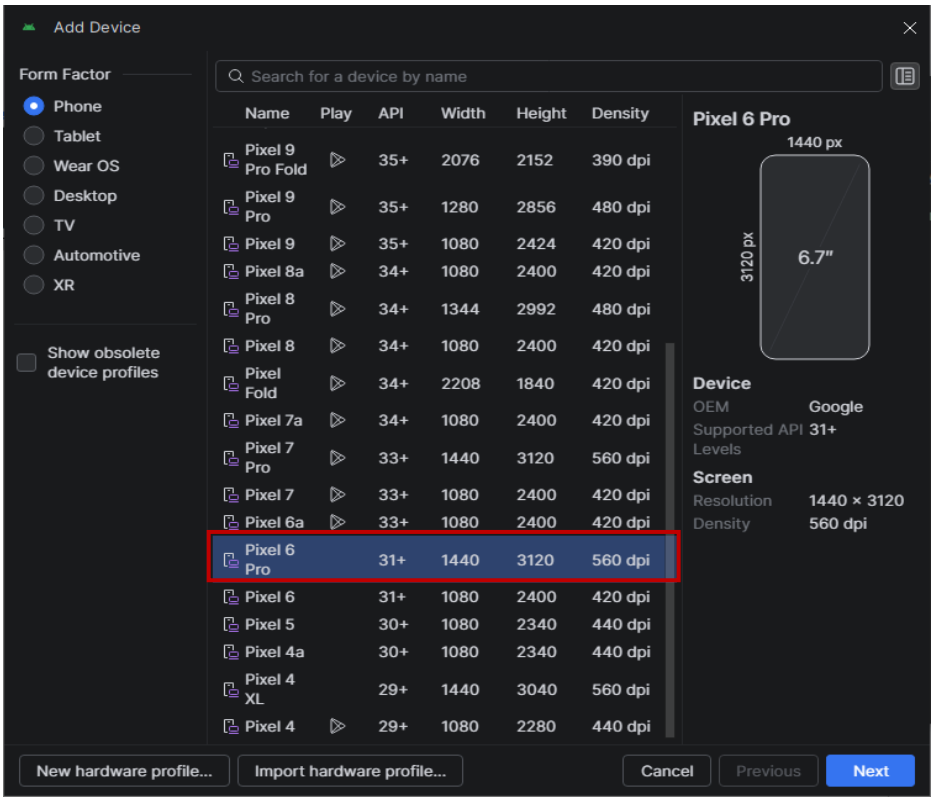


Figure 25: Android Studio Device Selection

created (as shown in figure 25).

After choosing the desirable devices, we chose **Android 14 API** and **Google API service. Android 14** is used for its **stability** compared to other latest versions, and **Google API** is chosen because it allows **root access**, which the “Google Play” option will restrict [7]. Installing the system image might be needed before completing this task. The device is named after the student ID of 1 member, as required (as shown in figure 26).

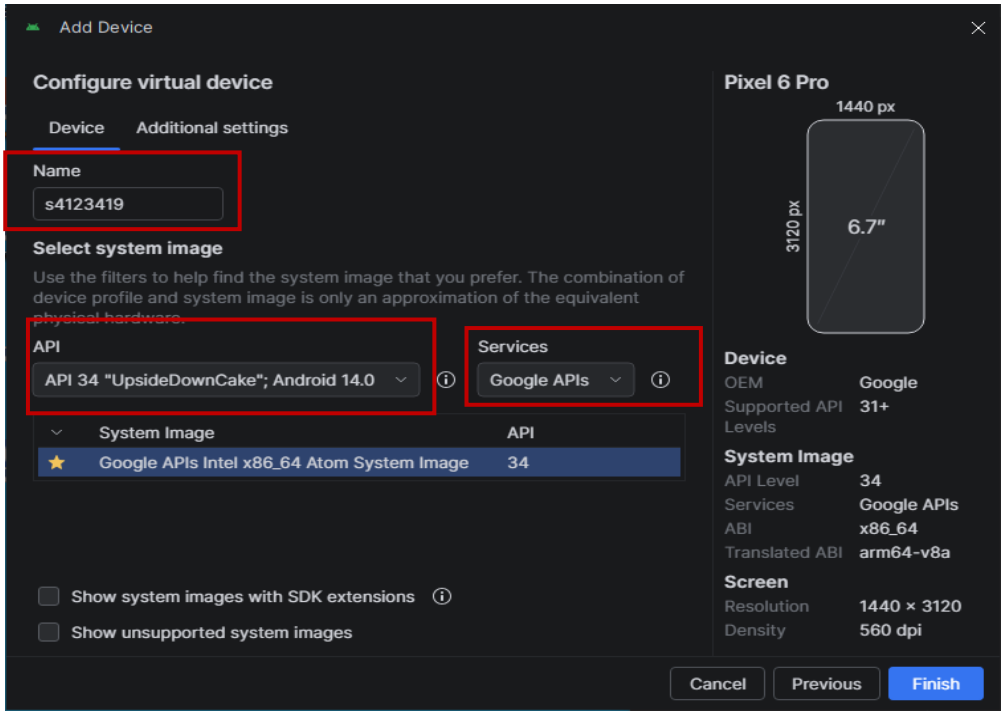


Figure 26: Android Studio Device Configuration

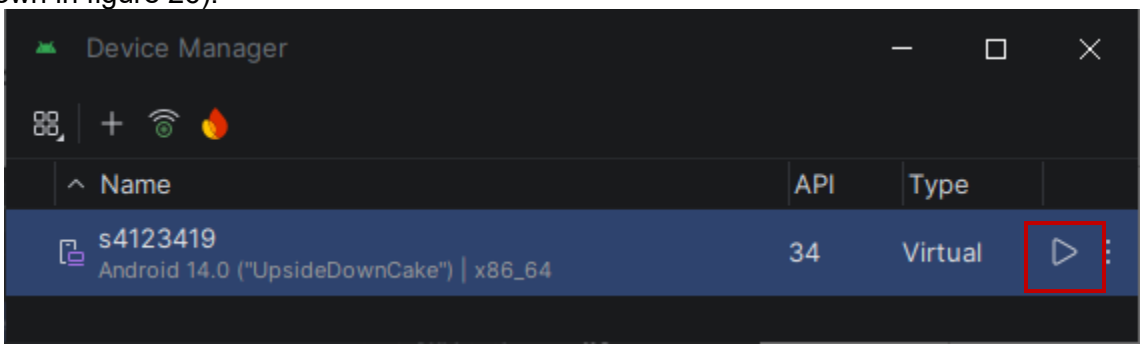


Figure 27: Virtual Device successfully created.

With that being done, the device will be created and shall appear in the device manager. By clicking on the play button **“▷”**, the device is booted (as shown in figure 27).

If successful, a device will be displayed on the desktop (as shown in figure 28).

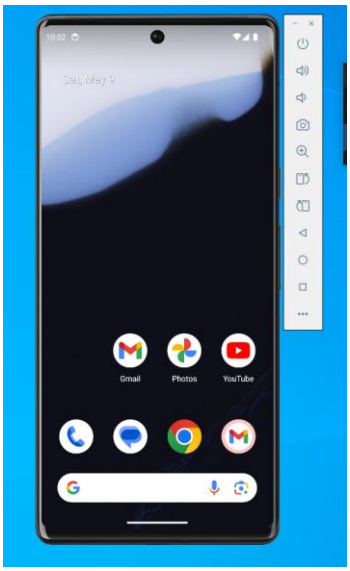


Figure 28: The emulator of Google Pixel 6 Pro

b. Create Contacts and Messages

To create contacts, we used the cursor to try to mimic the finger flick motion to open the **application menu** and select **Contacts** (As shown in figure 29).

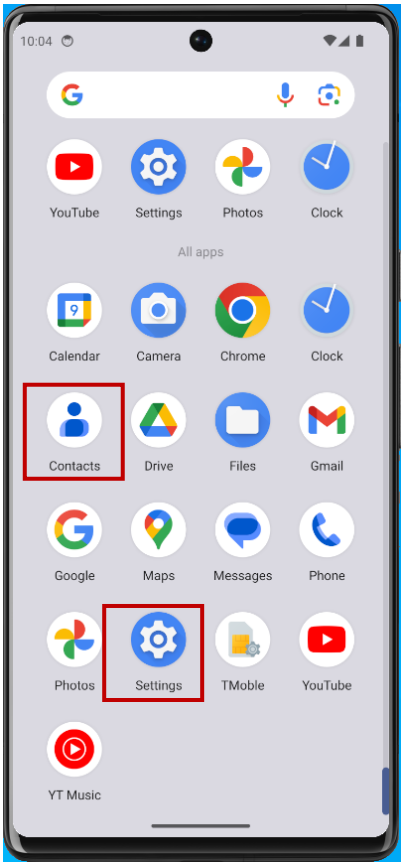


Figure 29: Pixel 6 Pro application menu

Select the **“+”** button to create a contact (as shown in figure 30).

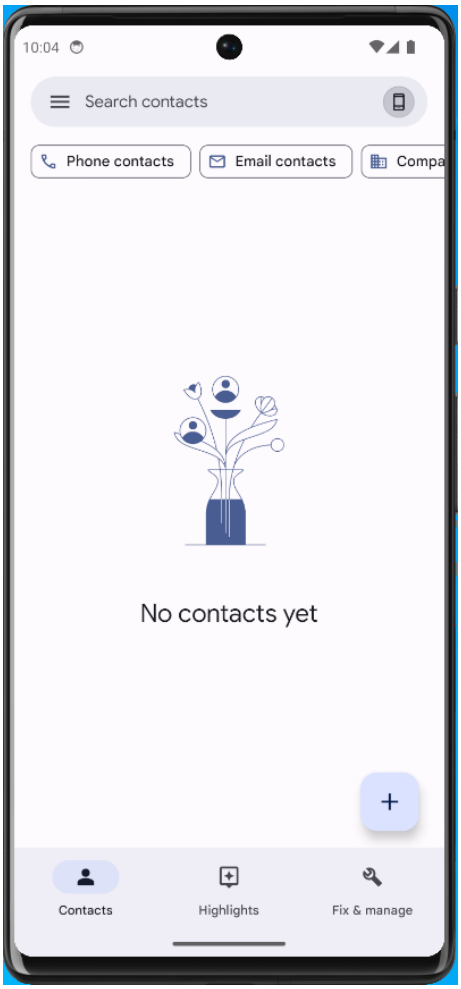


Figure 30: Phone’s Contacts

Here we set the contact's name to one member's name and set the phone number as **04XXXXXXX1**. Then, we repeat the step with the name of another member and the phone number as **04XXXXXXX2** (with XXXXXXX replaced by our student number (as shown in figure 31, figure 32).

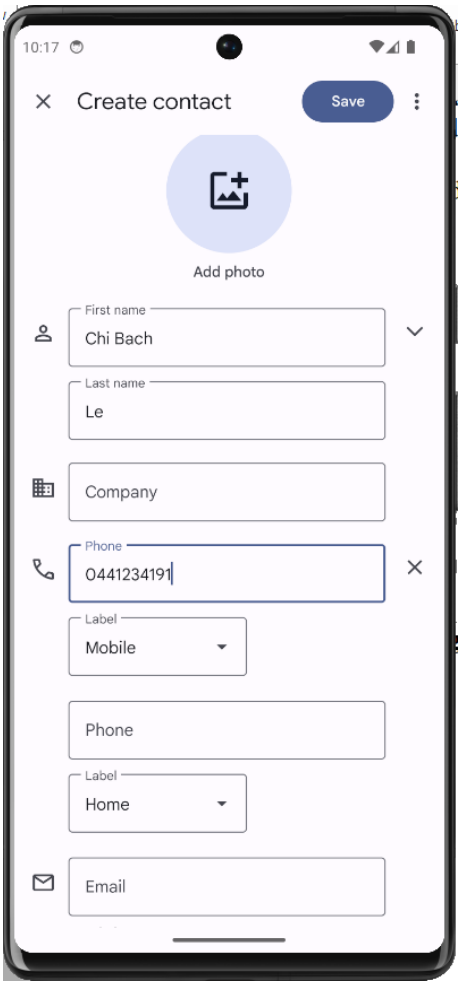


Figure 31: Creating contact

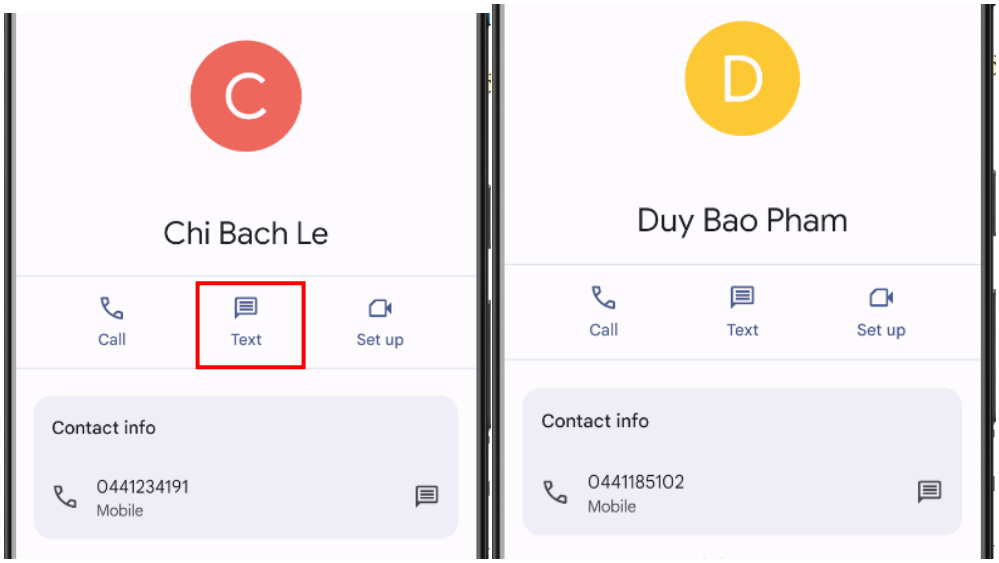


Figure 32: Created contacts

For the next step, we click on the **"Text"** option to be redirected to the Message application and send the messages to the right contact. (As shown in figure 33)

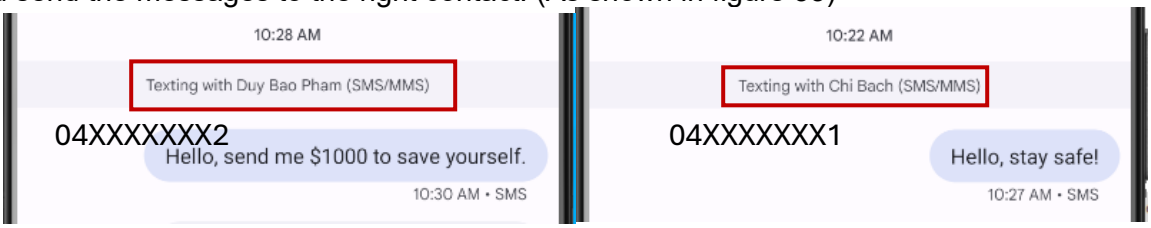


Figure 33: Created SMS

Part 2:

c. Data acquisition using ADB tool

To perform data acquisition on an Android device using ADB tool, the first step is to enter **developer mode** and **enable debugging mode**.

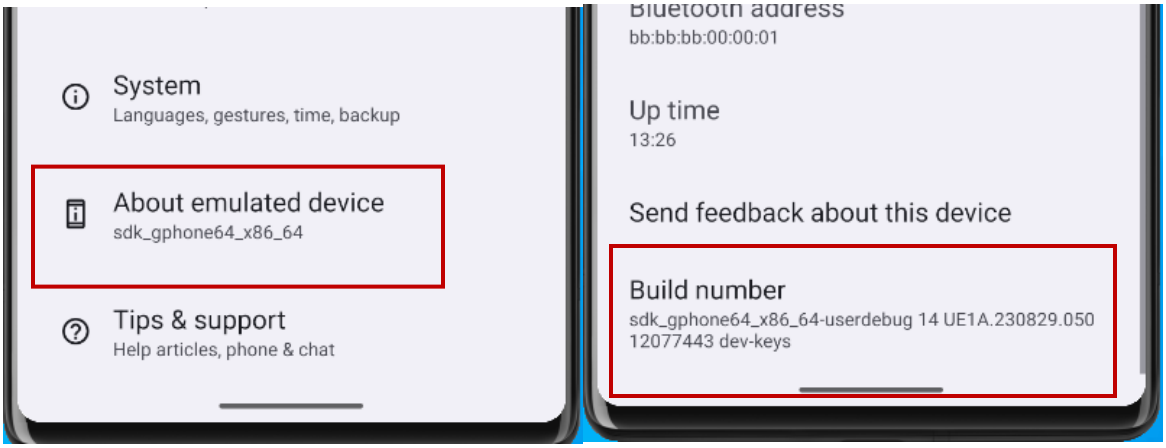


Figure 34: Enabling developer mode

To turn on developer mode, return to application menu (as shown in figure 29), select **Settings** > **About emulated device**. Scroll down and **click Build number 7 times** until there is a message saying: **"You are now a developer"** (because the virtual device was not running smoothly, we were unable to capture the message on the screen). (As shown in figure 34)

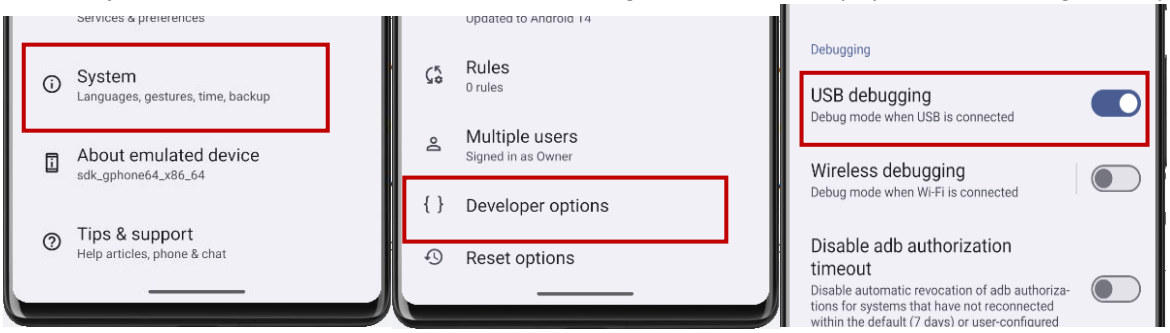


Figure 35: Enabling USB Debugging

Next, we return to **Settings**, select **System** then **Developer options**, and **enable USB Debugging**, which will grant permission for the virtual device to communicate with the host device through **Android Debug Bridge (ADB)** (as shown in figure 35).

ADB is a command-line tool that allows communication with a device. It provides access to a Unix shell that you can use to run a variety of commands on a device [6].

To use Android Debug Bridge, first we need to verify that ADB is installed. ADB is often installed with Android Studio. To verify its installation, we went back to the **Welcome window** (as shown in figure 23) of Android studio to open the **SDK Manager**.

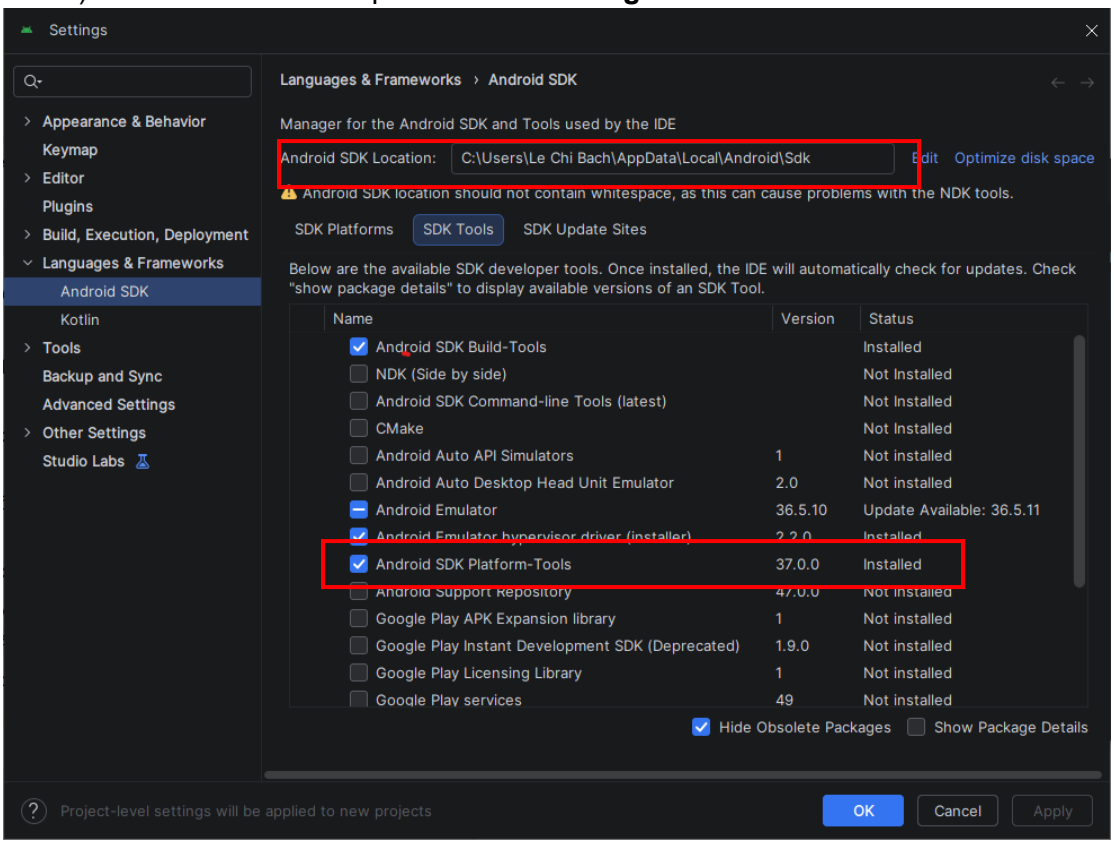


Figure 36: SDK manager

In the list of available tools, it could be seen that **Android SDK Platform-Tools** was checked and labeled as installed. This means that ADB has already been installed. We also took the **SDK location** and added **"\platform-tools"** to access ADB via Windows CLI (As shown in figure 36).

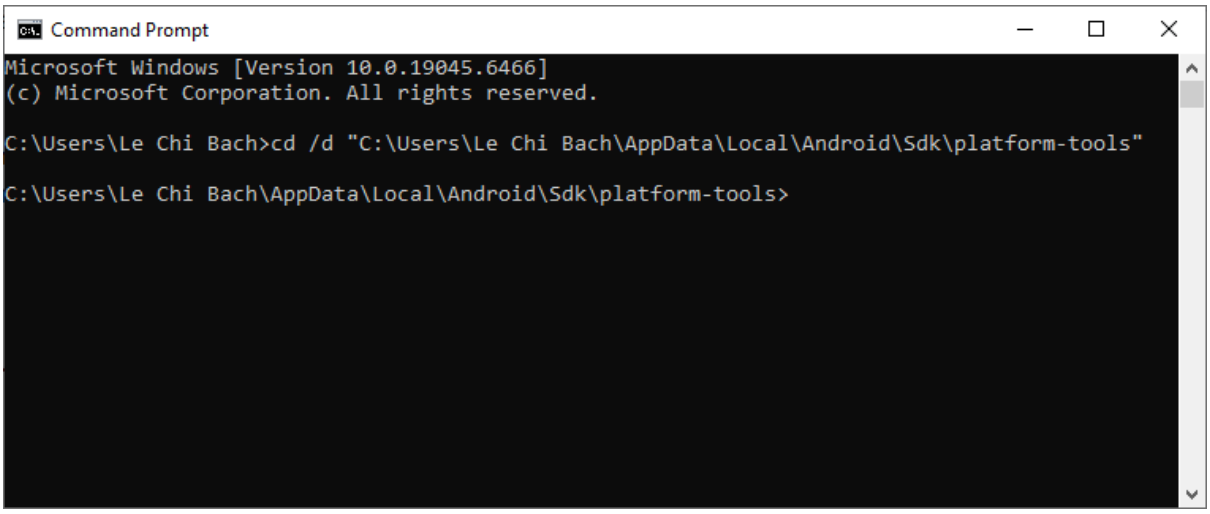


Figure 37: Accessing ADB using Window CLI

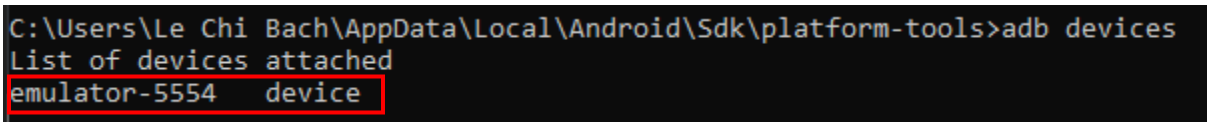


Figure 38: Running **"adb devices"** command

We used the **"adb devices"** command to show the **list of attached devices**. It showed one device which was the created emulated phone (As shown in figure 38).

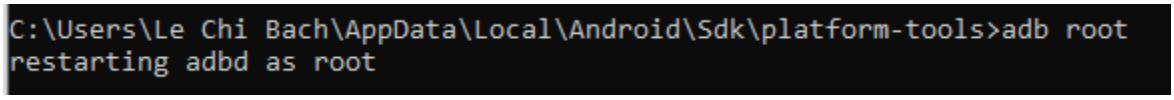


Figure 39: Running **"adb root"** command

Next, we run the command **"adb root"**. The returned message was **"restarting adbd as root"**, which meant the **daemon** (the component which runs commands on a device) had been restarted with **root permission** (privileged access) [7]. (As shown in figure 39)

d. Identify the locations and names of the SMS and Contact databases

The SMS database is often named **"mmssms.db"**[10] and the contacts database is often named **"contacts2.db"**[11] The next step was to locate the database on the device. We used **"adb shell"** to access the phone system, then used the **"find /data -name "msssms.db"** to locate the exact location of the SMS database. The same is done for **"contacts2.db"**.

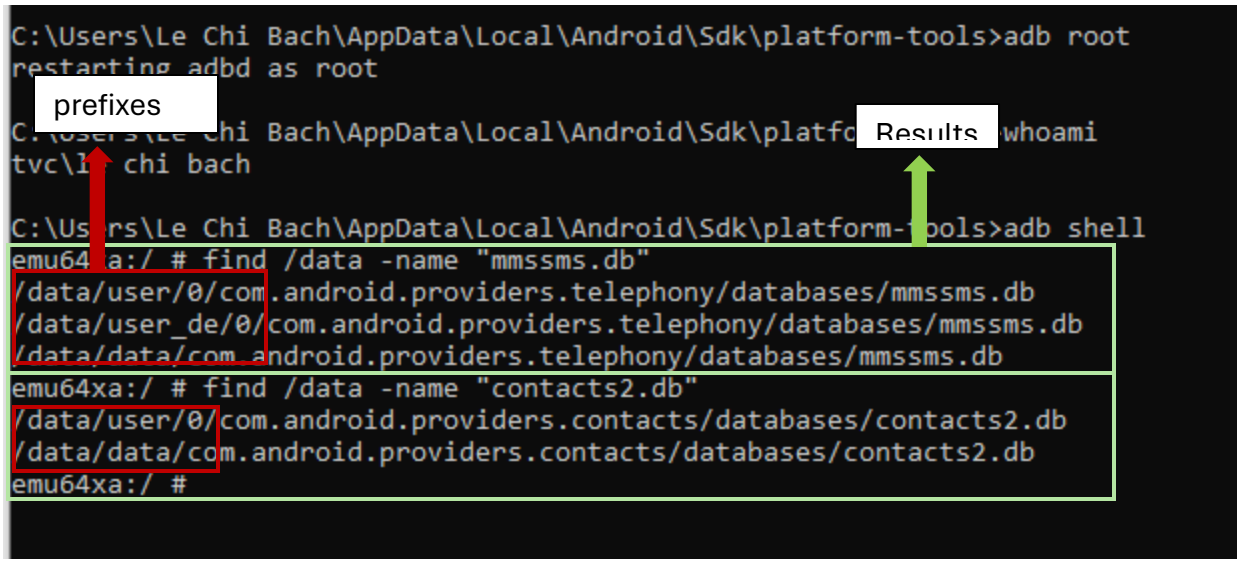


Figure 40: the location of the SMS and Contacts Databases

The results showed 3 results for **"mmssms.db"** and 2 results for **"contacts2.db" (**As shown in figure 40). They had different prefixes pointed to different directories. **"data/data"** is where the app databases and settings are stored, and the needed databases will be stored in the **"data"** folder [12]. We expected to find the required databases here. **"data/user/0"** is the system user with the ID of 0 in an android device that allows multiple users. It often belongs to the main user and is a symlink to **"data/data"** [13] means the 2 paths would likely point to the same directory. **"data/user_de/0"** stores data in device encrypted (de) storage, which is the storage location for direct boot mode (and after the user unlocked the device as well) [14]. By default, apps don't run in direct boot mode. Apps need to be registered to run in this mode. For example, notifications or timers are tasks that run when the device is not unlocked [15]. Messages and contacts list are not the content that can be accessed in direct boot mode (when the device is locked), but they sometimes can still be stored in this partition [14]. Therefore, the databases are not guaranteed to be found in **"data/user_de/0"**. We will attempt to find the databases in the data folder as this is the location where the databases are normally stored [12] **"data/data/com.android.providers.telephony/databases/mmssms.db"** and **"data/data/com.android.providers.contacts/databases/contacts2.db"** were the commands used to copy the databases to the computer.

e. Copy the databases to the computer.

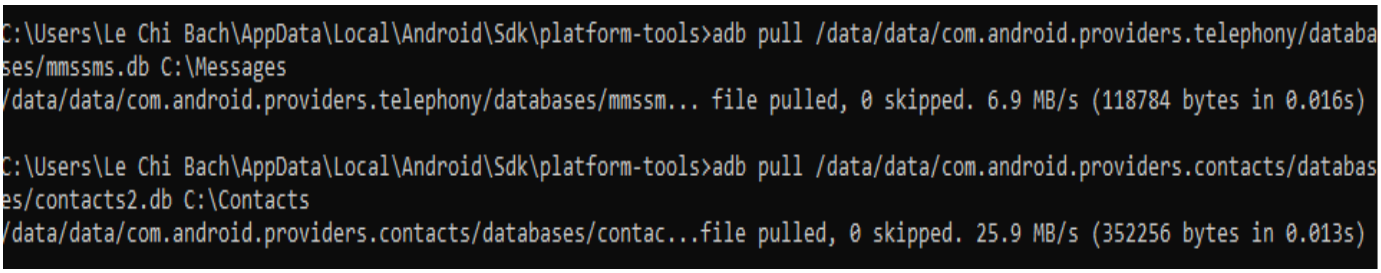


Figure 41: Copying SMS database and Contacts database to the computer

To copy the databases to the computer, we used the **"adb pull [source directory] [destination directory]"** command. The complete command would be **"adb pull data/data/com.android.providers.telephony/databases/mmssms.db C:\Messages"** and **"adb pull data/data/com.android.providers.contacts/databases/contacts2.db C:\Contacts"** (As shown in figure 41). After running the 2 commands, we opened the folders specified in the command, and we saw one database in each. (As shown in figure 42, figure 43)

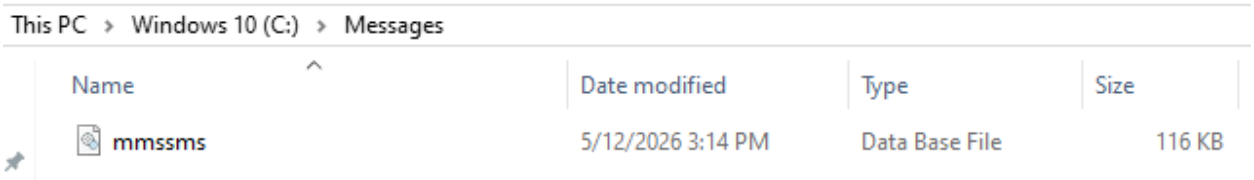


Figure 42: SMS database copied to the computer

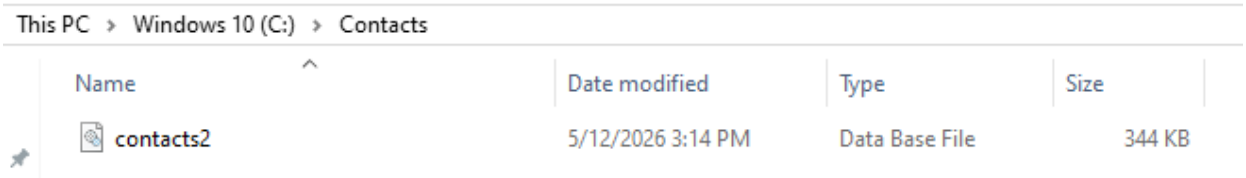


Figure 43: Contacts database copied to the computer

f. Access permission issue

During the implementation of **Question 3-part e**, we did not encounter any **"Access permission"** issue. The reason for that was possibly because we have run the **"adb root",** which allowed running command with high privilege, in **Question 3-part c**. We tried to run the **"adb pull"** command again without root access, and the returned message was **"Permission denied"**. The result (As shown in figure 44) had confirmed our suspicion for the reason why we did not encounter the forementioned problem. In addition, it is important that **Google API** service is chosen in the creation of the device because the other option (**Google Play**) will not provide **elevated privilege**(root) [7].

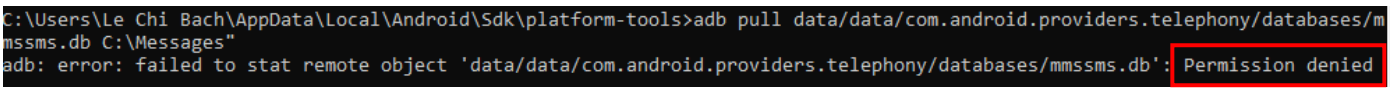


Figure 44: Running **"adb pull"** requests without root permission

g. Retrieve the SMSs and the contact details.

After copying the databases to our computer successfully, we can examine them to see the contacts and messages that we have created in the emulator.

In this task, we used **SQLite Viewer** to read the acquired databases.

After uploading the **mmssms.db**, we found the messages in the **“sms” table**, with the receivers' numbers (As shown in figure 45).

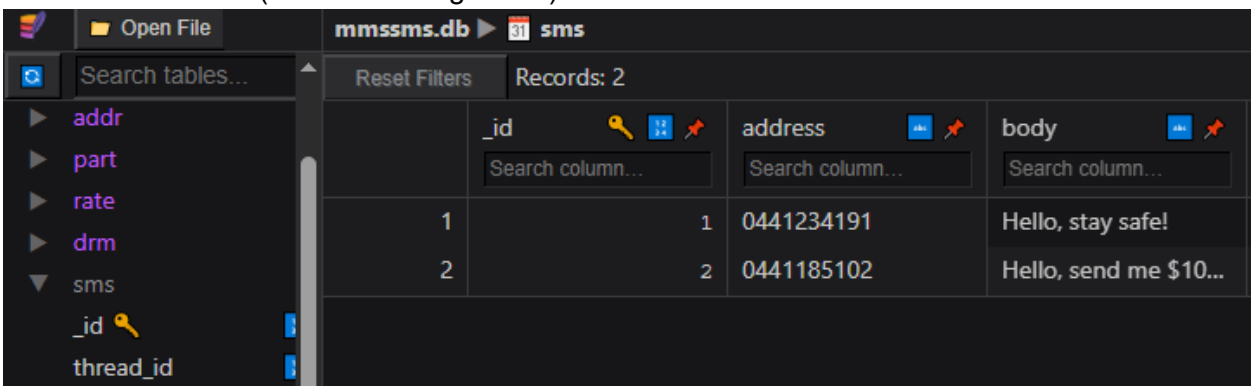


Figure 45: **mmssms.db** database, **“sms”** table

The result matched exactly with the contents we created in the emulator.

We also uploaded **contacts2.db** to examine.

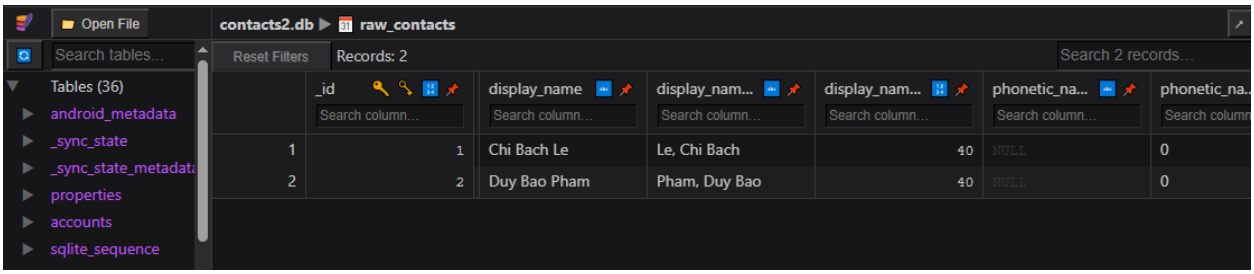


Figure 46: **contact2.db** database, **“raw_contacts”** table

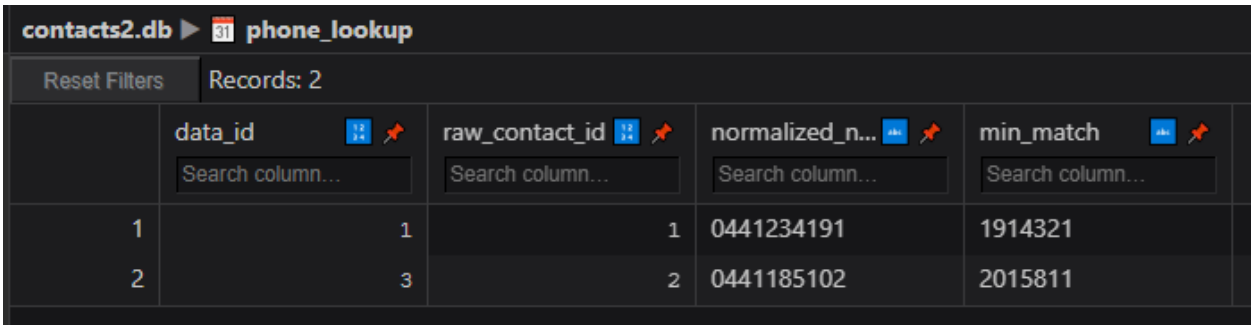


Figure 47: **contact2.db** database, **“phone_lookup”** table

In the **contacts2.db**, we found the contact name in the **“raw_contacts”** table, and the phone numbers in the **“phone_lookup table”**, with the **“raw_contact_id”** attribute matching the number with the equivalent **“_id”** attribute in the **“raw_contacts”** table. (As shown in figure 46, figure 47)

h. Conclusion

Question 3 has demonstrated the methodology of digital forensics on a mobile device, focusing on data acquisition and analysis. In the completion of the task, we have obtained some key insights that we thought would be relevant in a real-world scenario. Extracting artifacts from core system applications strictly requires root access. To perform data acquisition, bypassing production security is a critical step, which is shown in the difference between **Google APIs** and **Google Play** system images. The demonstration of the task also highlights the operation of **File-Based Encryption (FBE)**. The presence of messages and contact databases in the credential encrypted location (**/data/user/0** or **/data/data)** instead of the device's encrypted location confirms that user data privacy is locked until the device is unlocked.

Q4. Digital Forensic Analysis and Preparing Investigation Report

Investigation Report

RMIT (University) Court

Case Name: Decryption of Suspect's Encrypted Artifacts and Analysis of Bank Details

Case No: CASE-A2-Q004

Report compiled by Pham Duy Bao and Le Chi Bach - Digital Forensic Analyst

Date of investigation: 31/03/2026

Date report compiled: 22/05/2026

Signed:

I declare that in submitting all work for this assessment I have read, understood and agree to the content and expectations of the Assessment declaration.

DOCUMENT CONTROL

Version	Date	Description of Change	Author
01	22/05/2026	No change	Pham Duy Bao and Le Chi Bach

DISCLAMER

This report was prepared by Pham Duy Bao and Le Chi Bach as part of the requirements for the Computer and Internet Forensics course at RMIT University. The findings, tools, and analysis presented in this report are based on digital artifacts provided for academic purposes and are intended solely for educational and assessment use.

All information within this document is derived from a controlled forensic investigation of simulated case data (CASE-A2-Q004). No real-world data or personal information was involved. Any resemblance to actual events, individuals, or entities is purely coincidental.

This report is not intended for legal or commercial use and should not be distributed beyond its intended academic audience without permission.

I. Introduction

This investigative report presents the results of a comprehensive digital forensic analysis conducted for case number CASE-A2-Q004. The investigation was conducted after forensic experts collected additional digital artifacts from the suspect's external hard drive during a previous expanded investigation. In the previous case, evidence indicated the suspect's involvement in concealing information and data related to bank accounts. However, the newly discovered artifacts suggest the suspect may have been involved in a larger and more complex operation, involving data encryption, password concealment, and the use of cryptographic techniques to hinder forensic investigation.

The purpose of this investigation was to identify, analyze, and interpret the newly collected artifacts to determine whether they contained sensitive financial information and to verify the extent of the suspect's involvement in the case. Unlike the previous investigation, the files in the current artifact are encrypted using a more sophisticated method. Initial examination reveals that the decryption key or password is hidden in additional files, while the password file does not exist in plain text but is concealed or stored as a hash value. Furthermore, the hashing algorithm used is not readily apparent, requiring a cryptographic approach to determine the correct algorithm before proceeding with password recovery.

As the Digital Forensic Expert in this case, my assigned task is to continue analyzing the new artifacts to decrypt the hash values, recover the password, and use that information to access the protected files, thereby uncovering relevant banking information. Simultaneously, I need to evaluate and present how the recovered information proves or disproves the suspect's involvement in the suspected activities.

This report documents the entire forensic process, the tools used, the analytical methods, the evidence collected, and the results obtained during the investigation. Additionally, the report assesses the evidentiary value of the recovered data and analyzes the suspect's involvement based on the findings. The report's content is structured according to standard principles and procedures of digital forensic investigation to ensure clarity, systematic organization, and legal validity.

II. Executive Summary

1. Background & Scope.

The investigation was conducted under case reference **CASE-A2-Q004** following the recovery of additional digital artifacts named **EncryptedBankDetails.bin.** The analysis focused on identifying hidden credentials, determining the hashing algorithm used,

decrypting protected files, and recovering sensitive financial information potentially linked to the suspect.

The investigation was conducted in Window 11. The primary forensic tool utilized during the investigation included:

- **Autopsy**— used for digital evidence examination, deleted file recovery, file system analysis, metadata extraction, and artifact identification [16]
- **MD5Decrypt** — used as a rainbow-table-based hash lookup service to identify and recover the plaintext password associated with the recovered MD5 hash value [17]
- **FTK Imager** — FTK Imager is a tool for creating disk images and helping to preview data and for imaging [18].

2. Key Findings

The forensic analysis successfully uncovered a deleted text file called **aes_passwrd.txt** that held a cryptographic hash value. Using forensic analysis and cryptographic scrutiny, the hash was recognized as an MD5 hash due to its 32-character hexadecimal format.

The recovered hash value, **59270a7c20ca3edb7d32bf74ef707526**, was successfully decrypted with MD5Decrypt software, uncovering the plaintext password **johndoe775**

Additional forensic examination revealed two image files that included screenshots of **OpenSSL** commands for encryption and decryption, illustrating the application of **AES-256-CBC** encryption alongside salt protection.

Using the recovered password, investigators successfully decrypted the file **EncryptedBankDetails.bin**, which was later identified as a concealed image file containing additional encrypted information encoded using **Caesar Cipher** and **Morse Code** techniques.

3. Impact

The findings indicate that the suspect intentionally employed multiple layers of cryptographic protection. The evidence gathered showed that the suspect concealed sensitive information by encrypting key data, deleting password files, and obscuring the final content using classical cryptographic methods.

4. Recommendations

- Organizations and individuals should use stronger hashing algorithms such as SHA2 and SHA3 family along with salting mechanisms for better password protection

- Security monitoring solutions should be configured to detect suspicious use of encryption tools such as OpenSSL, particularly when encryption is performed on sensitive financial or organizational data.
- Perform frequent forensic readiness evaluations for systems managing sensitive financial data.

III. Scope and Objective of the Engagement

1. Engagement Scope

The scope of this forensic engagement includes:

- Examination of the artifact file **A2_Q4_Artifact.zip**
- Recovery and analysis of deleted and allocated files
- Identification of cryptographic artifacts and hidden clues
- Identification of hashing algorithms
- Password recovery and decryption analysis
- Examination of encrypted financial files
- Documentation and preservation of forensic evidence

2. Engagement Objectives

The objectives of this investigation are:

- To identify hidden or deleted files related to password storage
- To determine the hashing algorithm used within the evidence
- To recover the plaintext password from the identified hash
- To decrypt the encrypted financial file
- To analyze additional clues hidden within recovered files
- To evaluate the suspect's involvement based on forensic findings

3. Changes & Value-Adds

This investigation extends the findings from the previous case by introducing:

- Password encryption using hashing algorithms
- Multi-layer decryption methodology
- Correlation of artifacts across multiple evidence files

IV. Evidence Verification and Integrity

To ensure the scientific validity and legal admissibility of the digital evidence, cryptographic hashing verification was performed prior to any analysis. The checksum of the forensic image

file was calculated and compared with the original checksum of the gold copy provided at the time of collection. This file is named **checksum.txt,** which details are shown in Appendix A.1.

The verification was conducted using the FTK Imager tool.

Evidence Verification Table

Artifact File Name	Expected Checksum (Provided)	Calculated Checksum (Actual)	Algorithm	Status / Verification Result
A2_Q4_Artifact.zip	4fa1df5f37b04500980c0ecdc1f52caf	4fa1df5f37b04500980c0ecdc1f52caf	MD5	**MATCHED** (Integrity Verified)
A2_Q4_Artifact.zip	129cbb0d901a81a5ea5b45230f6ad5c905101e32	129cbb0d901a81a5ea5b45230f6ad5c905101e32	SHA-256	**MATCHED** (Integrity Verified)

Forensic Conclusion on Integrity: The calculated hash values perfectly match the expected checksum provided in the **checksum.txt** file (as shown in Appendix A.2). This confirms with absolute mathematical certainty that digital evidence has not been altered, corrupted, or tampered during the transfer and acquisition phases. The integrity of the evidence is preserved, allowing the investigation to proceed.

V. Findings and Analysis:

1. Finding #1: Recovery of Hidden Password File (Medium Risk)

Using Autopsy software on the image file named A2_Q4_Artifact, investigators analyzed both the structure of the allocated and deleted files from the evidence image.

The following forensic navigation paths were utilized:

- **Deleted files:** File Views → Deleted Files → File System
- **Existing files:** File Views → File Types → By Extensions

During examination of deleted artifacts, a suspicious file named **aes_passwrd.txt** was identified. The file appeared to contain a cryptographic hash value potentially associated with encrypted data protection

Evidence Details

Name	Content	MD5	SHA256	MIME Type	Extension

aes_passwrd.txt	59270a7c20ca3edb7d32bf74ef707526 (1)	59270a7c20ca3edb7d32bf74ef707526	8dcef26b96b7f45f2d62d42b31c7a2d39cd3961a66debdaf19172e2ba6dff373	text/plain	txt

(1) The image is shown in Appendix A.3

2. Finding #2: Hash Identification and Password Recovery (High Risk)

The recovered value was analyzed based on standard hashing characteristics:

- One-way property [19]
- Fixed-length output [19]
- Avalanche effect [19]

Analysis suggests that the file ***aes_passwrd.txt*** may contain a **32-character hexadecimal** string, which is likely a password hash. Based on its fixed length characteristic, it is possibly an **MD5** hashing algorithm[16].

To retrieve the original plaintext value, investigators used MD5Decrypt, which functions by using rainbow table methods to compare known hash values with previously cataloged plaintext passwords[17].

Recovered Password

Hash Value	Algorithm	Recovered Password
59270a7c20ca3edb7d32bf74ef707526	MD5	johndoe775 (2)

(2) the result from the MD5Decrypt tool is shown in Appendix A.4

The successful password recovery demonstrated that the hash function was not salted, used outdated hashing algorithms, and the password was weak and vulnerable to rainbow table attacks.

3. Finding #3: Cryptographic Artifact Analysis (High Risk)

Additional examination revealed multiple image files with screenshots pertaining to encryption and decryption commands carried out by the suspect.

Recovered Cryptographic Evidence

Name	Description	MD5	SHA256	MIME Type	Flags (Dir)

Decrytion.png	OpenSSL decryption command (3)	f2337fc1341ce972aa6be69d7ecd3838	028454869bb84d7dc3e8d88fdc3b8abe55ab38bacb311b721af0f1c9c96b6630	image/png	Allocated
Encrytion.png	OpenSSL encryption command (4)	51bcf3d09eacc089008ce65e6fa3757d	da6c031c4d63484d652a7d91e8c8993a08bad1c6b93adf890b71d336b40b6c1e	image/png	Located
hashAlgo.jpg	Hashing algorithm clue (5)	011e1206a7ff4d6bd272a637590c8f04	0ac746785d7eab3a05833fb062ef5a3d84ec138a38f1acd0f704ed1594cbaab7	image/jpeg	Located
rotCipher.png	Caesar cipher clue (6)	820b1247bf7689c5f2a19d9310e2b78d	085808966ec056143a54958d3d4adb44afa6938c1e9120b8a062892738696bd6	image/png	Located
secretcode.png	Morse code clue (7)	c6a299d367fe828f70af8ca30fdf21b0	09d44101397c1d1fdbe538f017ee54a0fb68eecd1608175da5a7243a75c67f40	image/png	Located

Further supporting evidence corresponding to items (3) through (7) can be found in Appendix A.5 through Appendix A.9, respectively.

Encryption Method Analysis

Based on the evidence collected above, specifically the information from Decryption.png and Encryption.png, the content of the images includes:

- **openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.bin**
- **openssl enc -d -aes-256-cbc -in encrypted.bin -out decrypted.txt**

As evidenced by that information, several arguments regarding file EncryptedBankDetails.bin encryption methods have been put forward, including:

1. AES-256 encryption
2. CBC block cipher mode
3. Salt-based key derivation
4. OpenSSL encryption toolkit

4. Finding #4: Decryption of Financial Evidence File (Critical Risk)

Decryption process

To decrypt the file, OpenSSL tool was used with the following command "**openssl enc -d -aes-256-cbc -salt -in EncryptedBankDetails.bin -out decrypted.txt**", along with the password recovered above johndoe775

After decrypting the file successfully, the file had no meaning as the output was decoded by Base64 (as shown in Appendix A.10). However, Subsequent file signature analysis identified the strings:

- **JFIF**
- **Photoshop 3.0**

These findings confirmed that the decrypted file was not a standard text document, but rather an image file[18]. To restore the file to its appropriate format, a PowerShell command was used to convert the recovered .txt file into a .JPEG image file. After the conversion process was completed successfully, the image was rendered correctly (as shown in Appendix A.11).

Based on the evidence identified during the analysis process, the image shown in Appendix A.6 was determined to contain text encrypted using the Caesar cipher method, while the content in Appendix A.8 was identified as Morse code. Further analysis of the Caesar cipher revealed that the encryption key used was **K = 7**.

Using this information, the encrypted content extracted from the converted image file was successfully decrypted. After decoding, the information might be as follows:

- **Name:** John Doe
- **PID:** 775
- **Password:** Phantom 07

V. Summary of the Forensics Report

The case involves new artifacts collected from the suspect's external hard drive, specifically the USB disk image **A2_Q4_Artifact.zip**, which contains highly sensitive information, such as bank details. Our task is to decode the hash, retrieve the password, and use it appropriately to find the bank details by systematically analyzing the clues provided across the files, unlocking each clue to help solve the next one. The investigation took place from March 31, 2026, to May 22, 2026, and conducted within a controlled Windows 11 forensic workstation environment using industry-standard forensic tools and procedures.

During the analysis of case A2_Q4_Artifact.zip, we discovered several pieces of evidence related to the case, including images of the encryption and decryption commands and password. The conclusion shows that the perpetrator attempted to erase evidence by deleting images related to the encryption and encryption of bank information in the evidence file **EncryptedBankDetails.bin** by deliberately hiding the correct file format and encrypting the information using **Caesar** cipher and **Morse** code.

VI. Conclusion and Opinion from the Analysis

1. Finding #1

The user attempted to delete a password-related file named **aes_passwrd.txt**, believing it had been permanently removed. However, the operating system only marked the file's storage space as unallocated and available for reuse, while the underlying data remained intact until overwritten. By using Autopsy to analyze the forensic image, investigators were able to examine deleted artifacts and successfully recover the suspicious password file.

2. Finding #2

There are three characteries of the hashing algorithm which are one-way property, fixed-length output, and avalanche effect. In this case, the fixed-length output was used to identify the hashing algorithm of the password. With the **32-Hex v**alue, the **MD5** hashing algorithm was potentially used.

After identifying the hashing algorithm, investigators applied a rainbow table approach to recover the original plaintext password. Rainbow tables contain precomputed hash values associated with known plaintext passwords, allowing rapid comparison against recovered hashes. In this case, the **MD5Decrypt** tool was utilized to perform the hash lookup. Since this is an outdated hashing algorithm, doesn't use salting, and the password is quite weak, the process successfully recovered the password.

3. Finding #3

There are mostly 5 relevant images of the case that were discovered suggesting encryption techniques and methods. The analysis of these images suggests that the file **EncryptedBankDetails.bin** was potentially decrypted using the **OpenSSL** encryption toolkit with **AES-256** encryption in **CBC** mode and **salt-based** key derivation, corresponding to the parameters "**openssl**", "**-aes-256-cbc",** and **"-salt**" in the decryption.png.

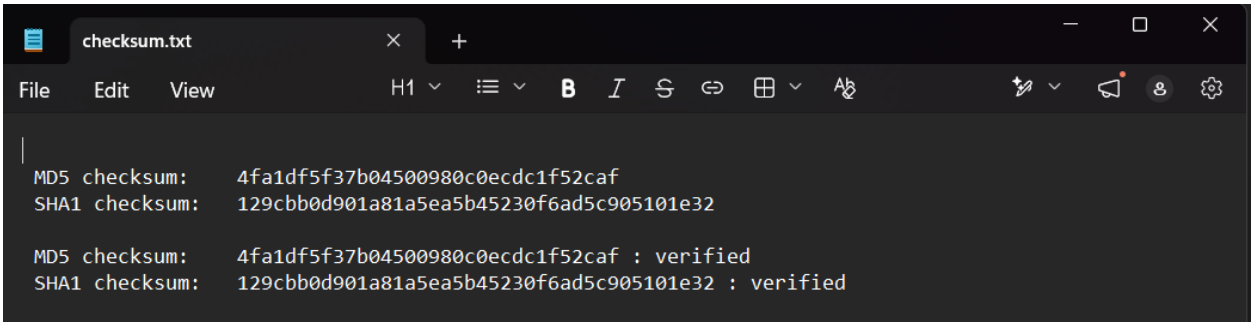
4. Finding #4

The investigation verified that the file **EncryptedBankDetails.bin** used **OpenSSL** for encryption with **AES-256-CBC** and a **salt** to generate the key. After successfully decrypting the file, the resulting data was identified as **Base64**-decoded image information. Transforming the decrypted data revealed an image file containing additional hidden information.

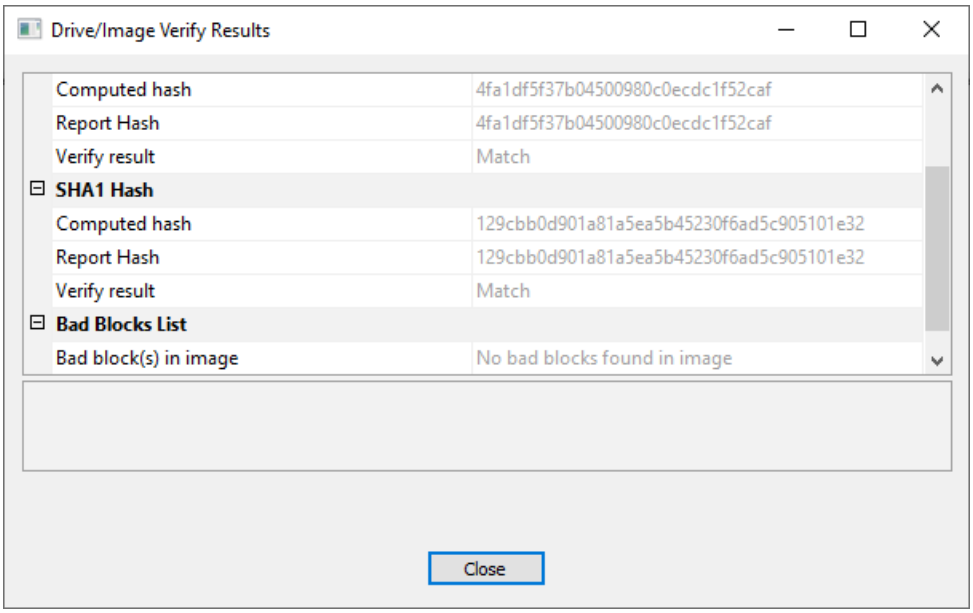
Further forensic analysis showed that the image content was secured using a combination of **Caesar cipher** and **Morse code**. By identifying the **Caesar** cipher shift key **K = 7**, some hidden information, such as the individual's name and PID, could be recovered. This indicates that multiple layers of encryption and encryption methods were intentionally used to conceal sensitive data and hinder simple forensic analysis. From an investigative perspective, the evidence indicates a moderate level of technical sophistication by the individual who created or concealed the data.

VII. Appendix

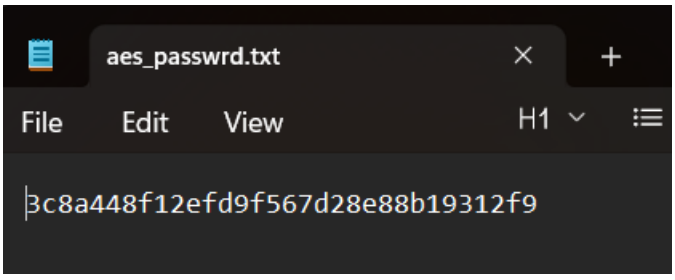
Appendix A.1. Details of the Provided Checksum File (checksum.txt)



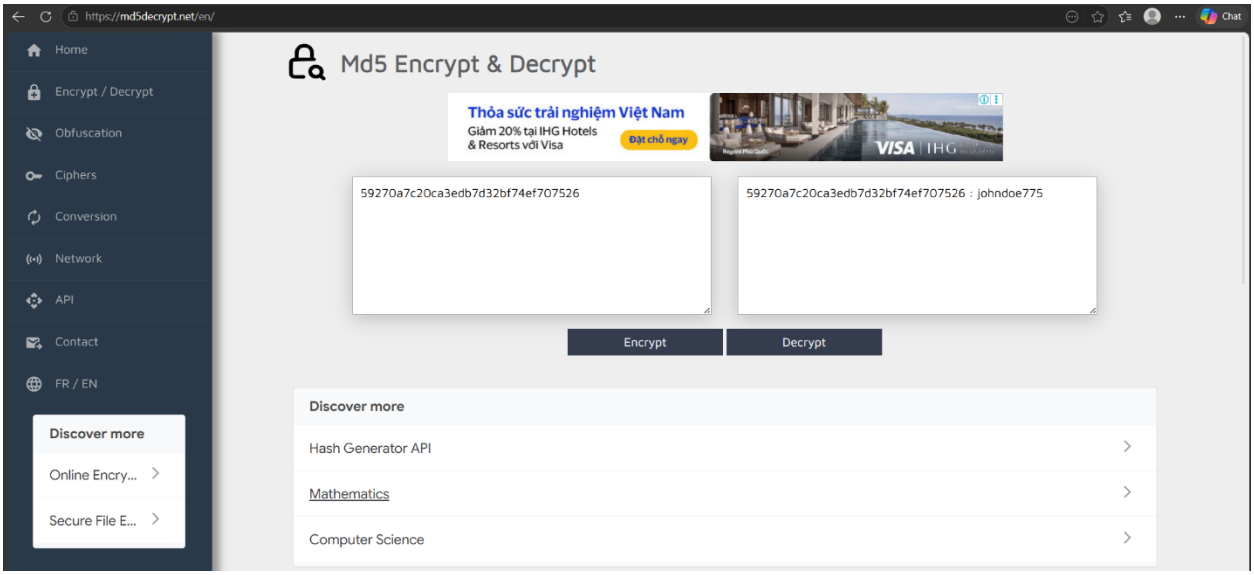
Appendix A.2 Checksum verification result from evidence.001 using FTK Imager



Appendix A.3 Recovered MD5 Hash Value from aes_passwrd.txt



Appendix A.4 Plaintext Password Recovered from the Hash Value Using the MD5Decrypt Tool



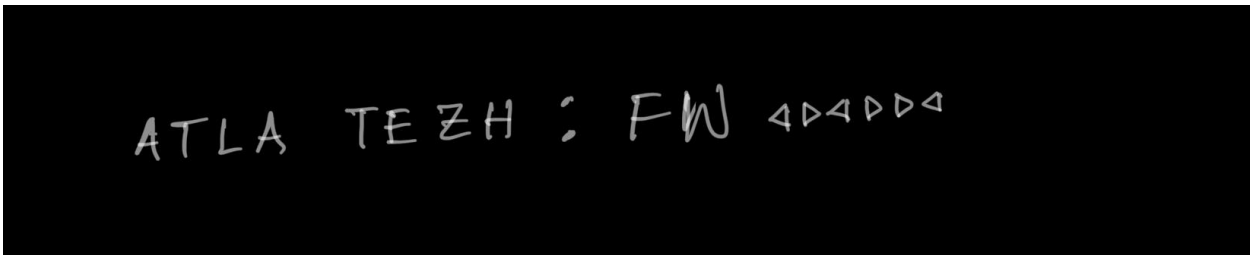
Appendix A.5 OpenSSL Decryption Command Recovered from Suspect Artifact

```
openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.bin
```

Appendix A.6 OpenSSL Encryption Command Recovered from Suspect Artifact

```
openssl enc -d -aes-256-cbc -in encrypted.bin -out decrypted.txt
```

Appendix A.7 Hashing Algorithm Clue Identified During Forensic Analysis



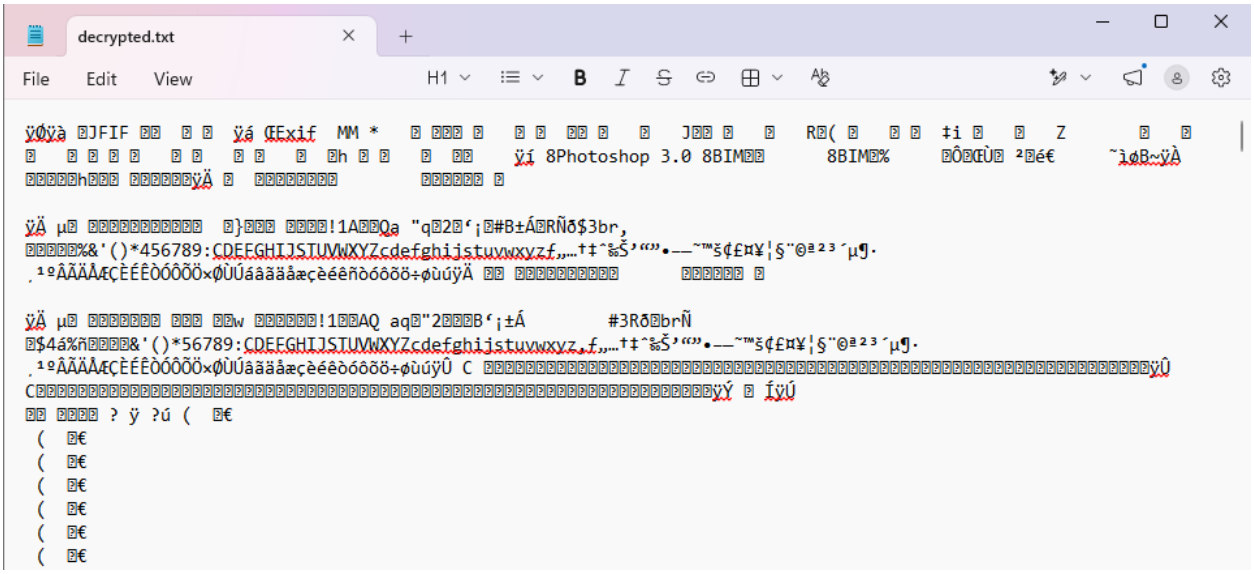
Appendix A.8 Caesar Cipher Clue Extracted from Recovered Evidence



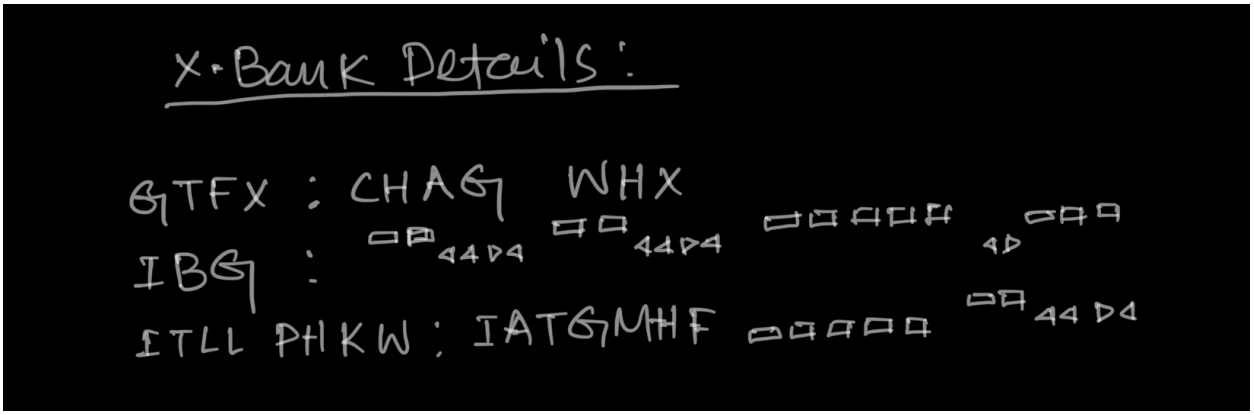
Appendix A.9 Morse Code Artifact Identified in Recovered File

a ·−	i ··	q −−·−	y −·−−	] −·−−	7 −−···
b −···	j ·−−−	r ·−·	z −−··	; −−−·	8 −−−··
c −·−·	k −·−	s ···	“ −···−	1 ·−−−−	9 −−−−·
d −··	l ·−··	t −	(−−·−−	2 ··−−−	0 ·····
e ·	m −−	u ··−	) ··−··−	3 ···−−	
f ··−·	n −·	v ···−	{ −·−−·	4 ····−	
g −−·	o −−−	w ·−−	} ·−···−	5 ·····	
h ····	p ·−−·	x −··−	[−−−·−	6 −····	

Appendix A.10 Base64-Decoded Output from the Decrypted File Name EncryptedBankDetails.bin.



Appendix A.11 Restored PNG Image Recovered After File Conversion



VIII. References

[1] GeeksforGeeks, “TCP 3-Way Handshake Process.” Accessed: May 15, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/tcp-3-way-handshake-process/

[2] Cloudfare, “SYN flood attack.” Accessed: May 15, 2026. [Online]. Available: https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/

[3] Murat Alagoz, “Change your User Agent in web pentests and bug bounties: Don’t Be a Plain Tester!” Accessed: May 15, 2026. [Online]. Available: https://mrtalagoz.medium.com/change-your-user-agent-in-web-pentests-bug-bounties-dont-be-a-plain-jane-98b442a0c601

[4] “Multipurpose Internet Mail Extension (MIME) Protocol - GeeksforGeeks.” Accessed: May 17, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/multipurpose-internet-mail-extension-mime-protocol/

[5] “RFC 5280 - Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile.” Accessed: May 19, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5280#section-4.1.2.2

[6] “Meet Android Studio | Android Developers.” Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/studio/intro

[7] “Create and manage virtual devices | Android Studio | Android Developers.” Accessed: May 16, 2026. [Online]. Available: https://developer.android.com/studio/run/managing-avds

[8] “Android Debug Bridge (adb) | Android Studio | Android Developers.” Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/tools/adb

[9] “Rooting (Android) - Wikipedia.” Accessed: May 15, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Rooting_(Android)

[10] “Android Messaging Forensics – SMS/MMS and Beyond - Magnet Forensics.” Accessed: May 15, 2026. [Online]. Available: https://www.magnetforensics.com/blog/android-messaging-forensics-sms-mms-and-beyond/

[11] “Android Contacts.” Accessed: May 15, 2026. [Online]. Available: https://forensafe.com/blogs/Android_Contacts.html

[12] “applications - Where do Android apps store data? - Android Enthusiasts Stack Exchange.” Accessed: May 15, 2026. [Online]. Available: https://android.stackexchange.com/questions/47924/where-do-android-apps-store-data

[13] “Users for system developers.” Accessed: May 15, 2026. [Online]. Available: https://android.googlesource.com/platform/frameworks/base/+/master/core/java/android/os/Users.md

[14] “File-based encryption | Android Open Source Project.” Accessed: May 16, 2026. [Online]. Available: https://source.android.com/docs/security/features/encryption/file-based

[15] “Support Direct Boot mode | Security | Android Developers.” Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/privacy-and-security/direct-boot

[16] “Autopsy - Digital Forensics - Sleuth Kit Labs.” Accessed: May 15, 2026. [Online]. Available: https://www.autopsy.com/

[17] “Encrypt and Decrypt your MD5 hashes online.” Accessed: May 19, 2026. [Online]. Available: https://md5decrypt.net/en/

[18] “How to Create a Forensic Image with FTK Imager? - GeeksforGeeks.” Accessed: May 20, 2026. [Online]. Available: https://www.geeksforgeeks.org/ethical-hacking/how-to-create-a-forensic-image-with-ftk-imager/

[19] Dennis. Byrne, *Full stack Python security : cryptography, TLS, and attack resistance*. Manning Publications Co., 2021.

[20] “MD5 hash in Python - GeeksforGeeks.” Accessed: May 15, 2026. [Online]. Available: https://www.geeksforgeeks.org/python/md5-hash-python/

[21] “Understanding Rainbow Table Attack - GeeksforGeeks.” Accessed: May 15, 2026. [Online]. Available: https://www.geeksforgeeks.org/ethical-hacking/understanding-rainbow-table-attack/

[22] “The Metadata in JPEG files - Exiv2.” Accessed: May 15, 2026. [Online]. Available: https://dev.exiv2.org/projects/exiv2/wiki/The_Metadata_in_JPEG_files